Adversarial Search

CS5491: Artificial Intelligence

ZHICHAO LU

Content Credits: Prof. Wei's CS4486 Course
and Prof. Boddeti's AI Course

TODAY

Part 1: Deterministic Games

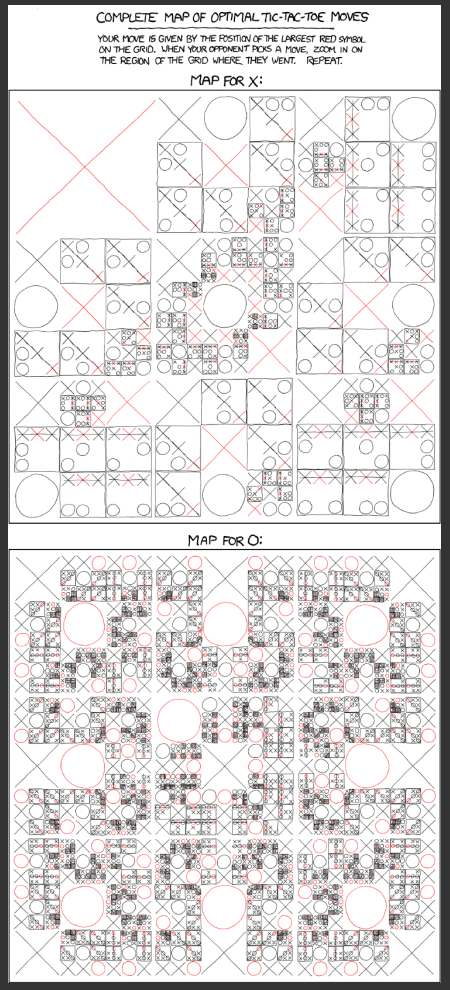
- Adversarial games, minimax search
- α-β pruning, depth-limited search

Part 2: Games with Chance

- Expectimax, utilities

Reading

- Today's Lecture: RN Chapter 5.1-5.3, 5.5, 16.1-16.3
- Next Lecture (Oct 8): Linear and Integer Programming



XKCD

AGENTS INTERACTING WITH AGENTS





Other agents act in the same world, with goals of their own: some cooperate, some compete. Games are the fully specified version of this.

GAME PLAYING STATE-OF-THE-ART

Checkers: 1950: first computer player. 1994: Chinook ends the 40-year reign of human champion Marion Tinsley, using a complete 8-piece endgame database. 2007: checkers solved: perfect play is a draw.



GAME PLAYING STATE-OF-THE-ART

Checkers: 1950: first computer player. 1994: Chinook ends the 40-year reign of human champion Marion Tinsley, using a complete 8-piece endgame database. 2007: checkers solved: perfect play is a draw.

Chess: 1997: Deep Blue defeats world champion Garry Kasparov in a six-game match: 200M positions per second, deep α-β search, a hand-tuned evaluation. Today's engines (Stockfish, Leela) add neural-network evaluation and are far beyond any human.



GAME PLAYING STATE-OF-THE-ART

Checkers: 1950: first computer player. 1994: Chinook ends the 40-year reign of human champion Marion Tinsley, using a complete 8-piece endgame database. 2007: checkers solved: perfect play is a draw.

Chess: 1997: Deep Blue defeats world champion Garry Kasparov in a six-game match: 200M positions per second, deep α-β search, a hand-tuned evaluation. Today's engines (Stockfish, Leela) add neural-network evaluation and are far beyond any human.

Go: $b > 300$: brute-force search fails, and no hand-written evaluation works. 2016: AlphaGo defeats Lee Sedol with Monte Carlo tree search plus learned policy and value networks. 2017: AlphaGo Zero and AlphaZero learn Go, chess and shogi from self-play alone.



GAME PLAYING STATE-OF-THE-ART

Checkers: 1950: first computer player. 1994: Chinook ends the 40-year reign of human champion Marion Tinsley, using a complete 8-piece endgame database. 2007: checkers solved: perfect play is a draw.

Chess: 1997: Deep Blue defeats world champion Garry Kasparov in a six-game match: 200M positions per second, deep α-β search, a hand-tuned evaluation. Today's engines (Stockfish, Leela) add neural-network evaluation and are far beyond any human.

Go: $b > 300$: brute-force search fails, and no hand-written evaluation works. 2016: AlphaGo defeats Lee Sedol with Monte Carlo tree search plus learned policy and value networks. 2017: AlphaGo Zero and AlphaZero learn Go, chess and shogi from self-play alone.

Imperfect information: 2017 and 2019: Libratus and Pluribus beat top professionals at no-limit poker. 2022: Cicero plays Diplomacy at human level, negotiating with a language model.



ADVERSARIAL GAMES



TYPES OF GAMES

Many different kinds of games.

Axes:

- Deterministic or stochastic?
- One, two, or more players?
- Zero sum?
- Perfect information (can you see the state)?



TYPES OF GAMES

Many different kinds of games.

Axes:

- Deterministic or stochastic?
- One, two, or more players?
- Zero sum?
- Perfect information (can you see the state)?

Want algorithms for calculating a strategy (policy) which recommends a move from each state.



DETERMINISTIC GAMES

Many possible formalizations, one is:



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)

DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)
- **Actions** A (may depend on player / state)



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)
- **Actions** A (may depend on player / state)
- **Transition Function** $S \times A \to S$



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)
- **Actions** A (may depend on player / state)
- **Transition Function** $S \times A \to S$
- **Terminal Test** $S \to \{t, f\}$



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)
- **Actions** A (may depend on player / state)
- **Transition Function** $S \times A \rightarrow S$
- **Terminal Test** $S \rightarrow \{t, f\}$
- **Terminal Utilities** $S \times P \rightarrow R$



DETERMINISTIC GAMES

Many possible formalizations, one is:

- **States** S (start at s_0)
- **Players** $P = \{1, \ldots, N\}$ (usually take turns)
- **Actions** A (may depend on player / state)
- **Transition Function** $S \times A \rightarrow S$
- **Terminal Test** $S \rightarrow \{t, f\}$
- **Terminal Utilities** $S \times P \rightarrow R$

Solution for a player is a policy: $S \rightarrow A$



GAME THEORY

Zero-Sum Games

General Games



GAME THEORY

Zero-Sum Games

- Agents have opposite utilities (values on outcomes)
- Lets us think of a single value that one maximizes and the other minimizes
- Adversarial, pure competition

General Games



GAME THEORY

Zero-Sum Games

- Agents have opposite utilities (values on outcomes)
- Lets us think of a single value that one maximizes and the other minimizes
- Adversarial, pure competition

General Games

- Agents have independent utilities (values on outcomes)
- Cooperation, indifference, competition, and more are all possible



ADVERSARIAL SEARCH

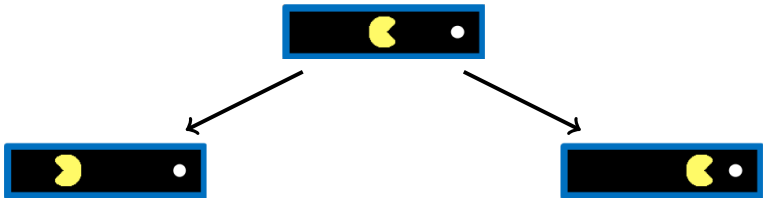


SINGLE-AGENT TREES

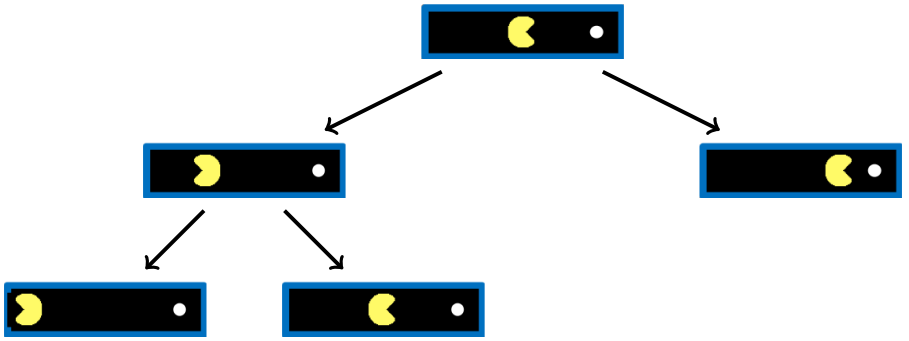
SINGLE-AGENT TREES



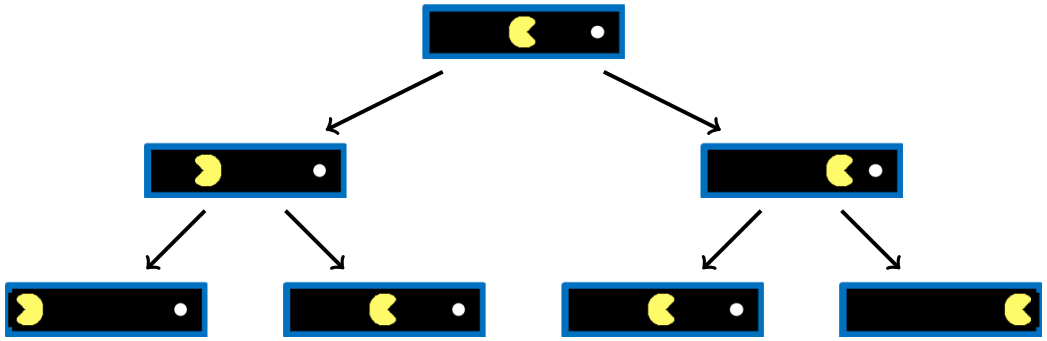
SINGLE-AGENT TREES



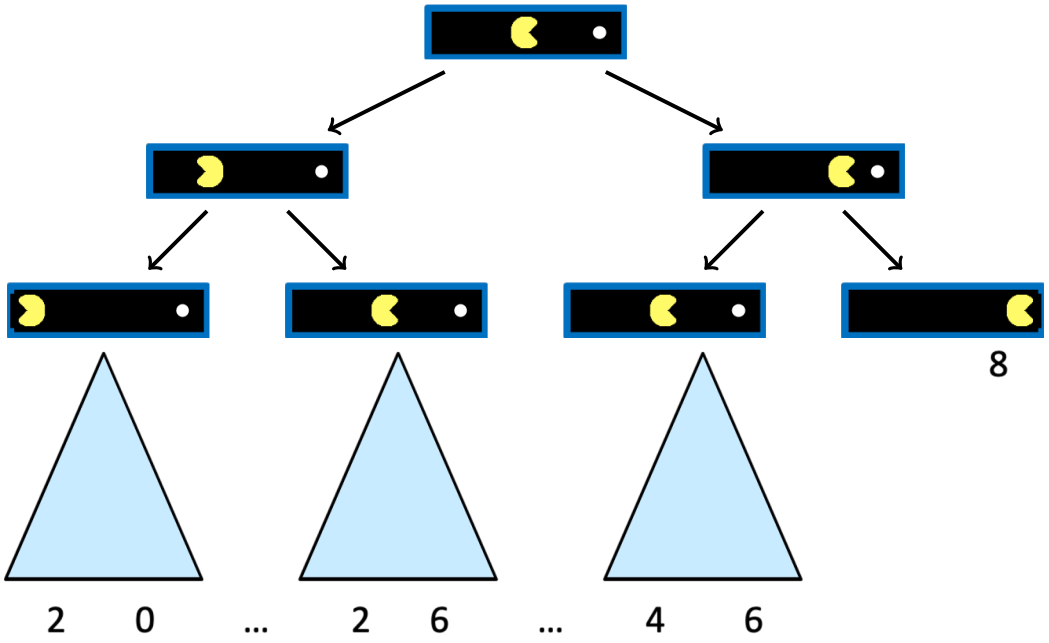
SINGLE-AGENT TREES



SINGLE-AGENT TREES

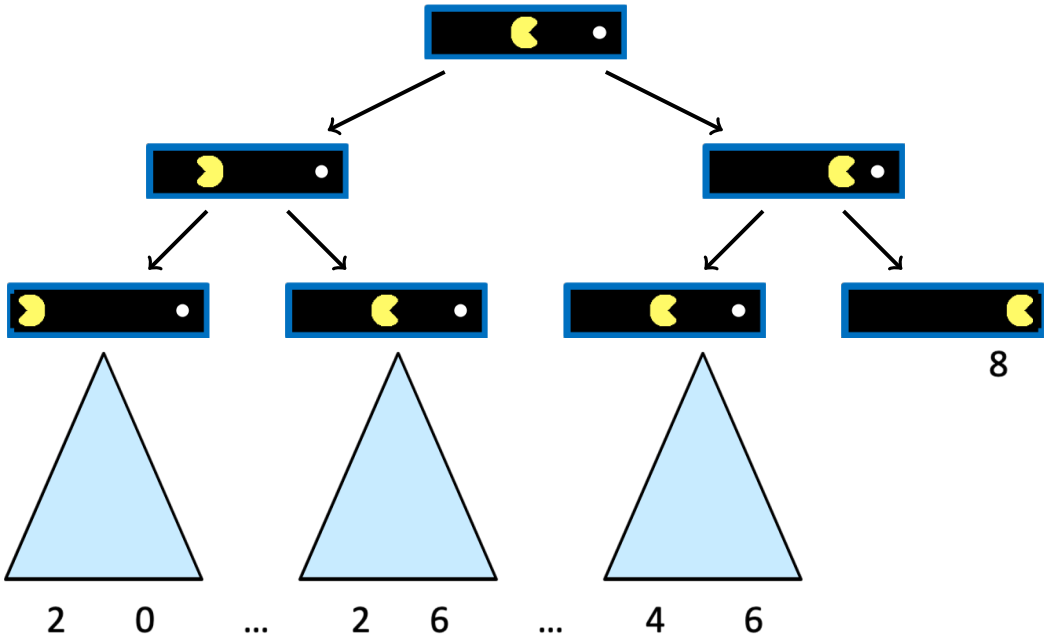


SINGLE-AGENT TREES



Value of State: The best achievable outcome (utility) from that state

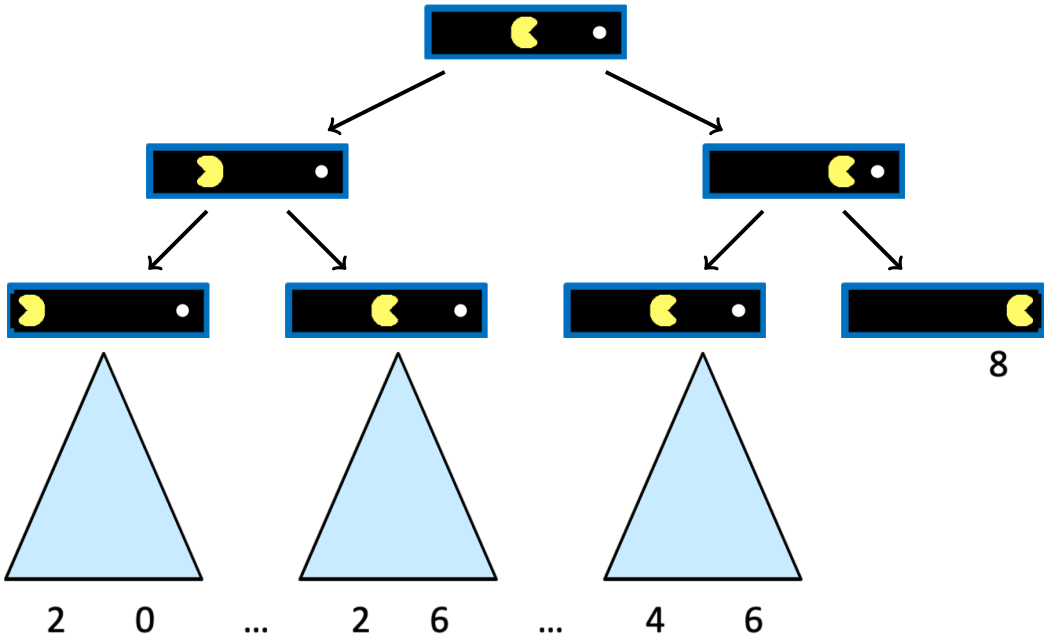
SINGLE-AGENT TREES



Value of State: The best achievable outcome (utility) from that state

Terminal States: $V(s)$ known

SINGLE-AGENT TREES



Value of State: The best achievable outcome (utility) from that state

Terminal States: $V(s)$ known

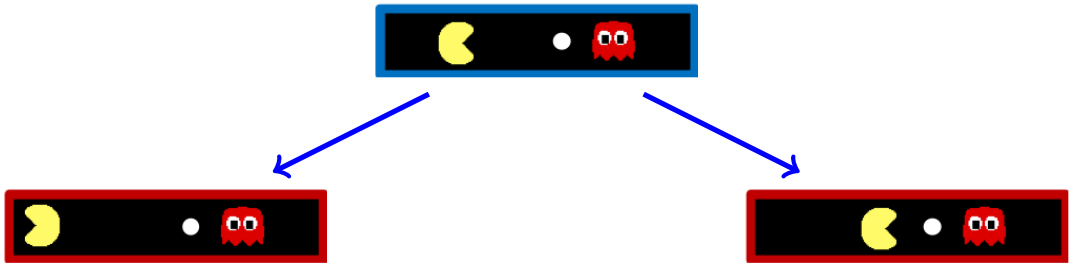
Non-Terminal States: $V(s) = \max_{s' \in children(s)} V(s')$

ADVERSARIAL GAME TREES

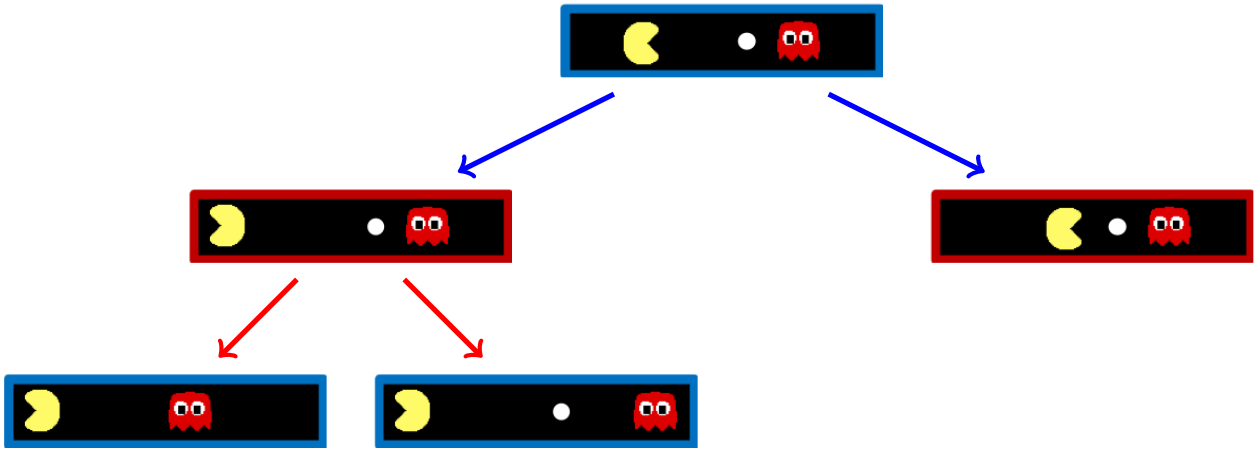
ADVERSARIAL GAME TREES



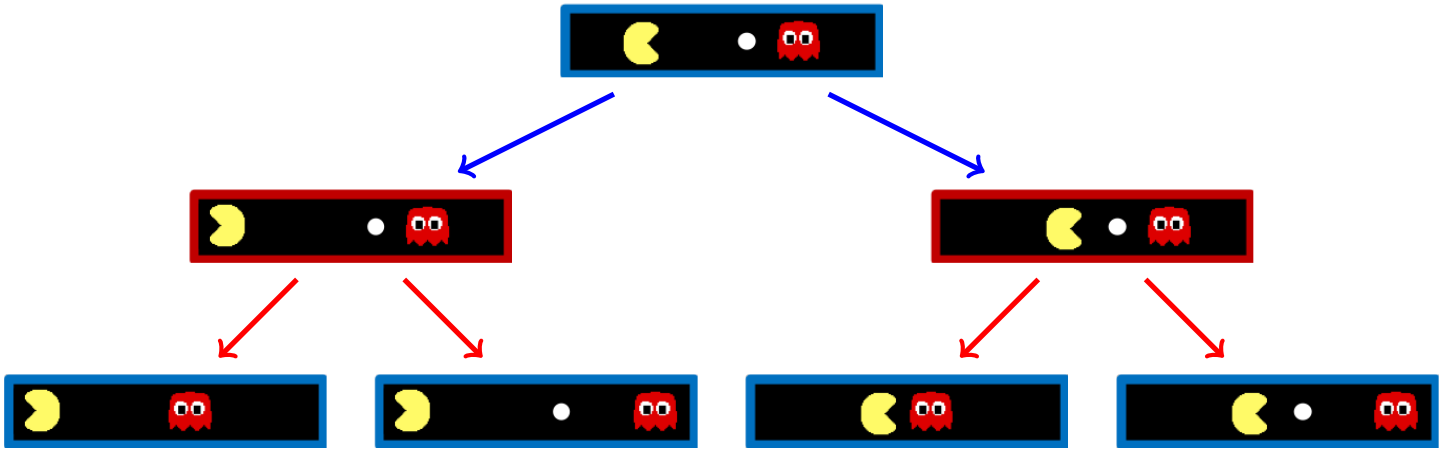
ADVERSARIAL GAME TREES



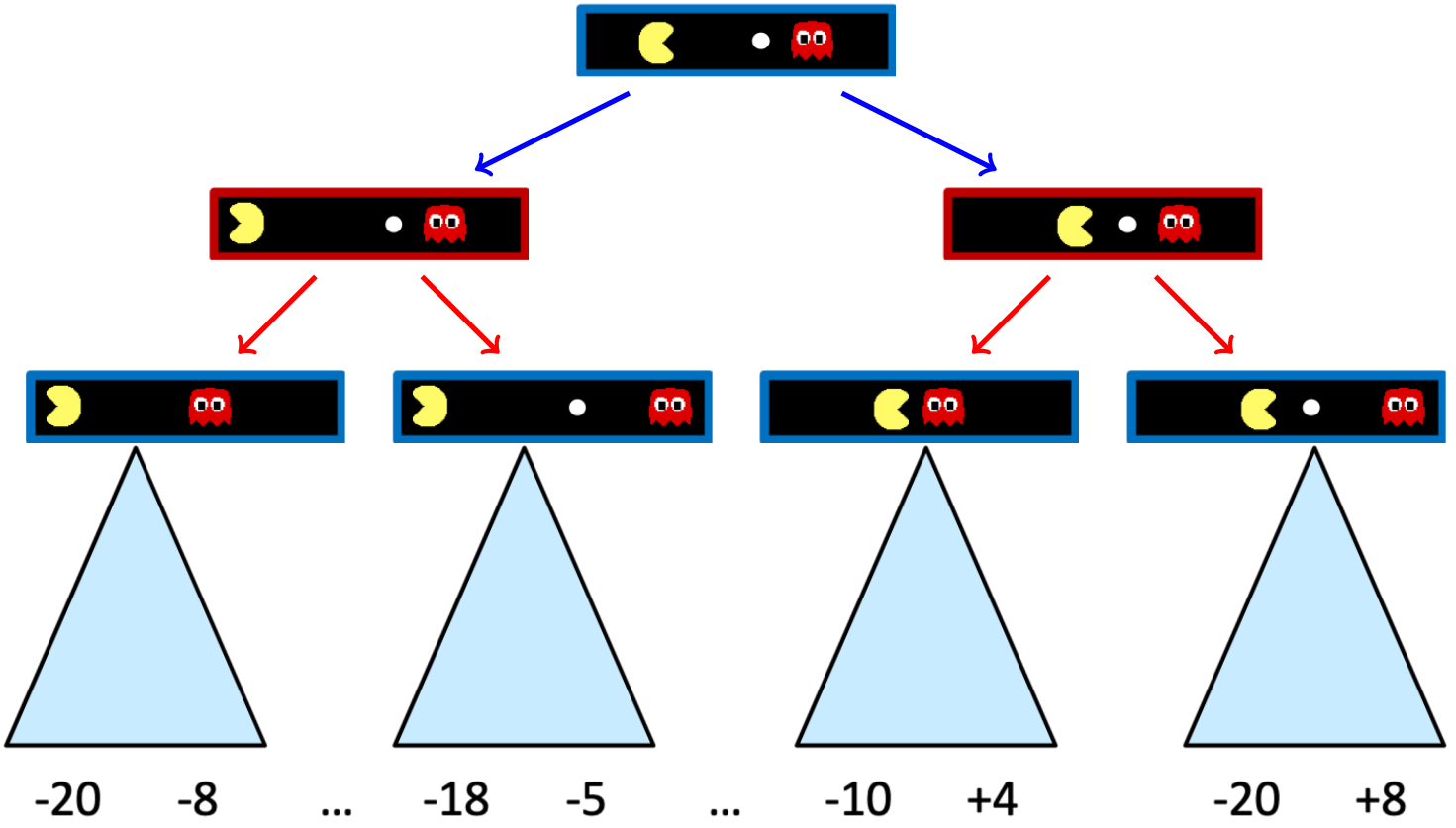
ADVERSARIAL GAME TREES



ADVERSARIAL GAME TREES



ADVERSARIAL GAME TREES



MINIMAX VALUES

MINIMAX VALUES

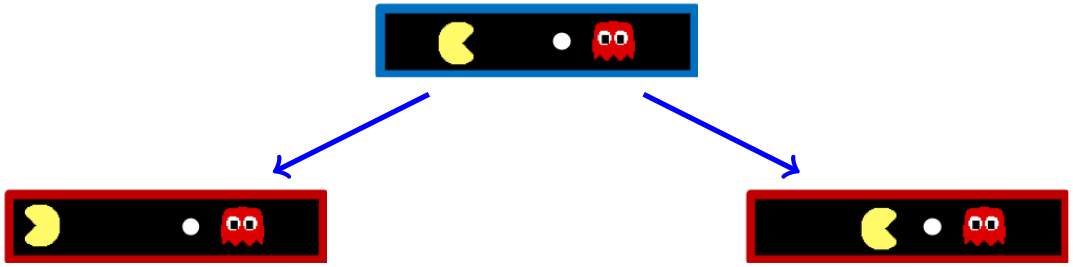


States Under Agent's Control: $V(s) = \max_{s' \in successors(s)} V(s')$

States Under Opponent's Control: $V(s) = \min_{s' \in successors(s)} V(s')$

Terminal States: $V(s)$ known

MINIMAX VALUES

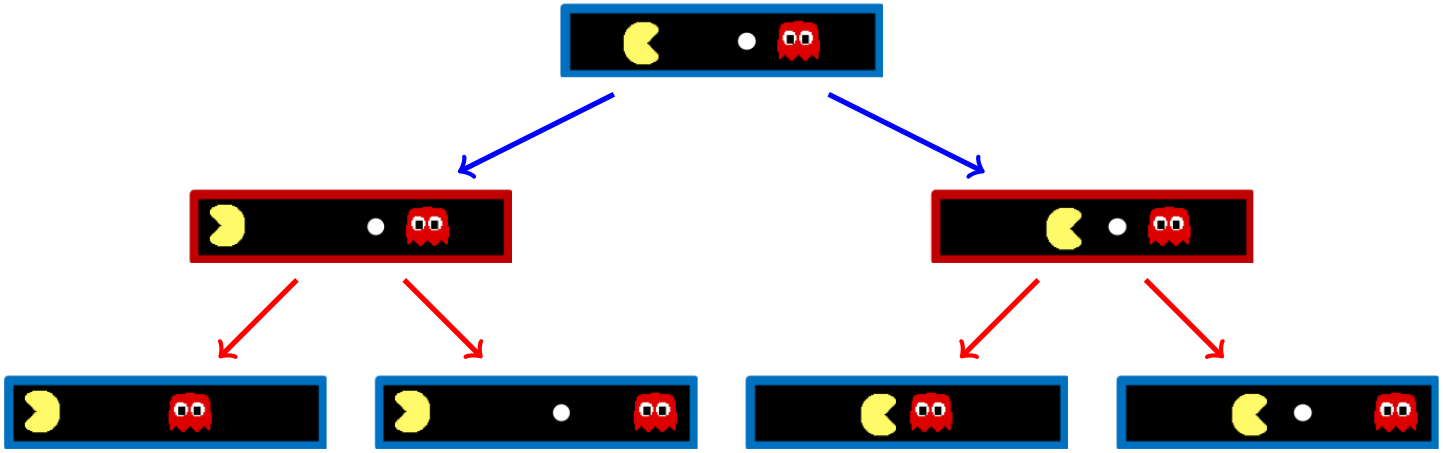


States Under Agent's Control: $V(s) = \max_{s' \in successors(s)} V(s')$

States Under Opponent's Control: $V(s) = \min_{s' \in successors(s)} V(s')$

Terminal States: $V(s)$ known

MINIMAX VALUES

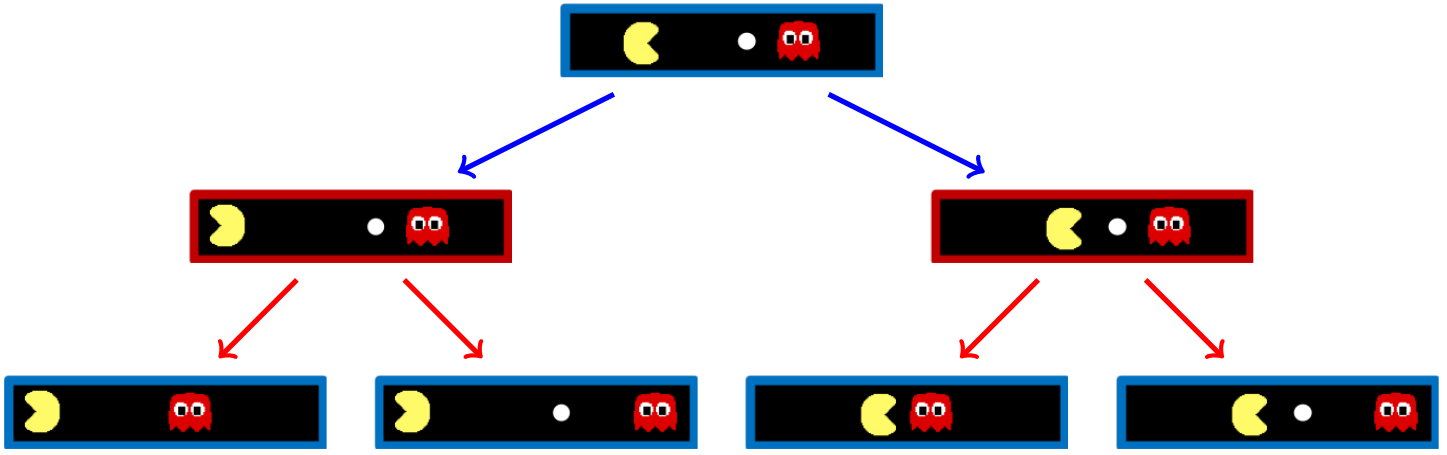


States Under Agent's Control: $V(s) = \max_{s' \in successors(s)} V(s')$

States Under Opponent's Control: $V(s) = \min_{s' \in successors(s)} V(s')$

Terminal States: $V(s)$ known

MINIMAX VALUES

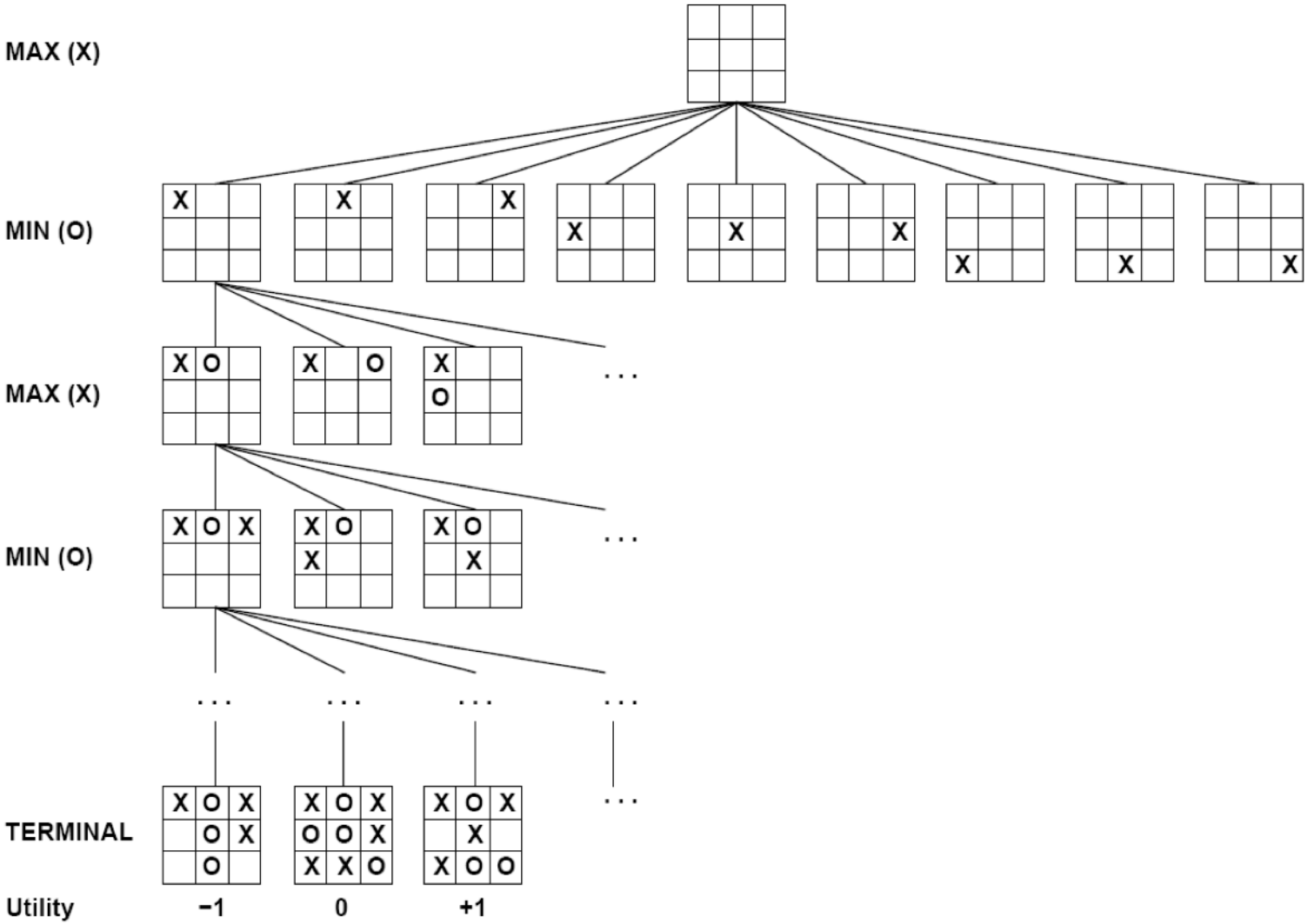


States Under Agent's Control: $V(s) = \max_{s' \in successors(s)} V(s')$

States Under Opponent's Control: $V(s) = \min_{s' \in successors(s)} V(s')$

Terminal States: $V(s)$ known

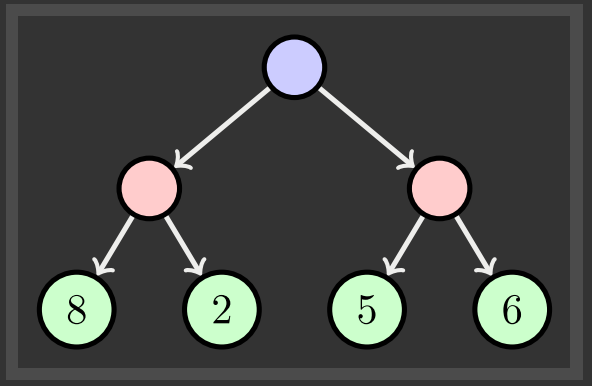
TIC-TAC-TOE



ADVERSARIAL SEARCH (MINIMAX)

Deterministic, zero-sum games:

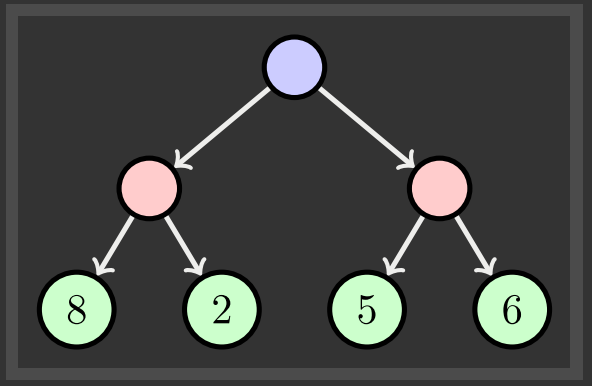
- Tic-tac-toe, chess, checkers
- One player maximizes result
- The other minimizes result



ADVERSARIAL SEARCH (MINIMAX)

Deterministic, zero-sum games:

- Tic-tac-toe, chess, checkers
- One player maximizes result
- The other minimizes result



Minimax Search:

- A state-space search tree
- Players alternate turns
- Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary

MINIMAX IMPLEMENTATION

MAX Agent:

Function *max-value(state)*
- $v \leftarrow -\infty$;
- **for** *each successor of state* **do**
  - $v \leftarrow \max(v, min - value(successor))$;
  - **return** v;
- **end**

MIN Agent:

Function *min-value(state)*
- $v \leftarrow +\infty$;
- **for** *each successor of state* **do**
  - $v \leftarrow \min(v, max - value(successor))$;
  - **return** v;
- **end**

MINIMAX IMPLEMENTATION (DISPATCH)

Dispatch:

```
Function value(state)
    if state is a terminal state then
        return the state's utility
    end
    if next agent is MAX then
        return max-value(state)
    end
    if next agent is MIN then
        return min-value(state)
    end
```

MAX Agent:

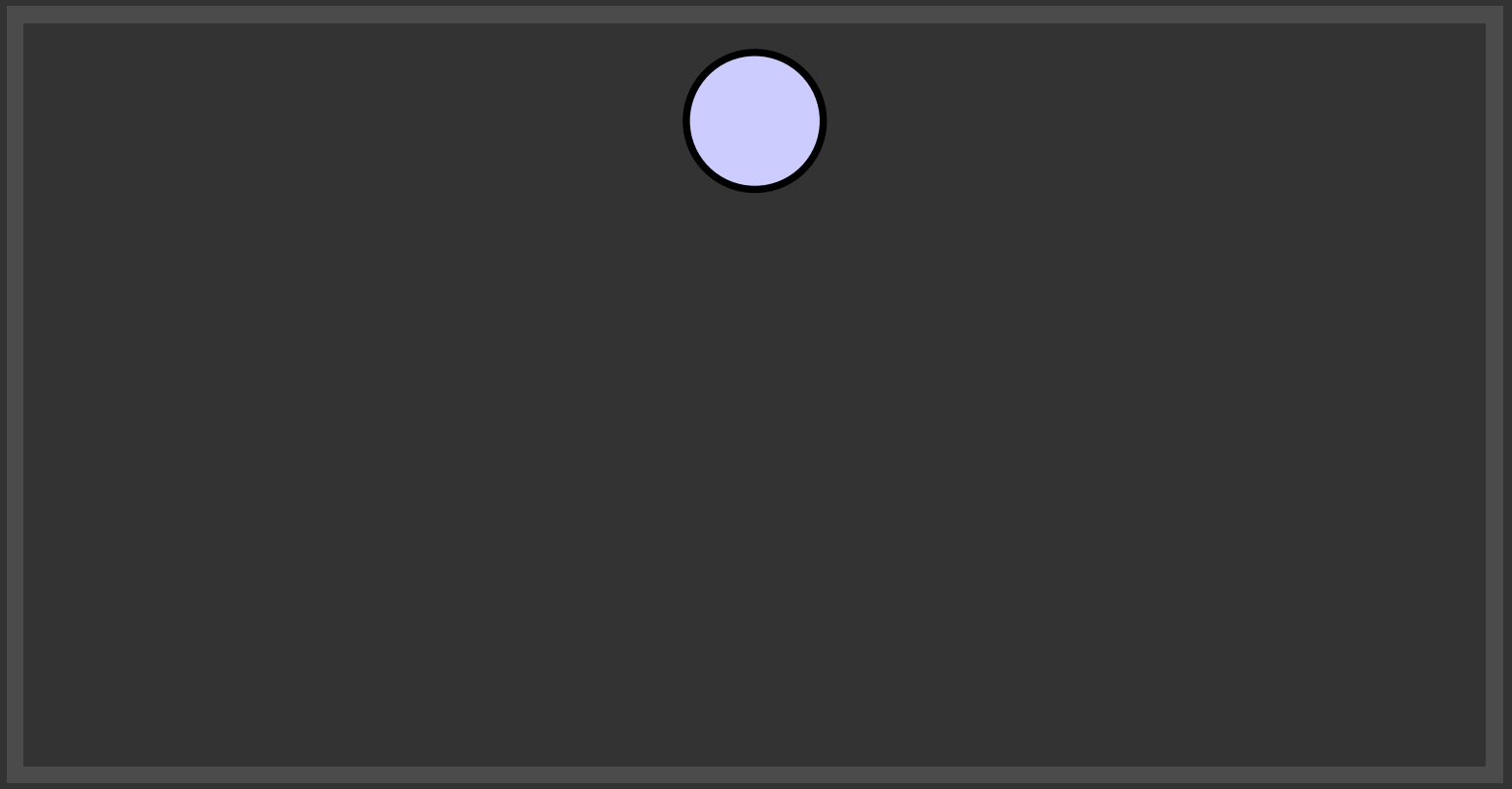
```
Function max-value(state)
    v ← −∞ ;
    for each successor of state do
        v ← max(v, min − value(successor)) ;
        return v;
    end
```

MIN Agent:

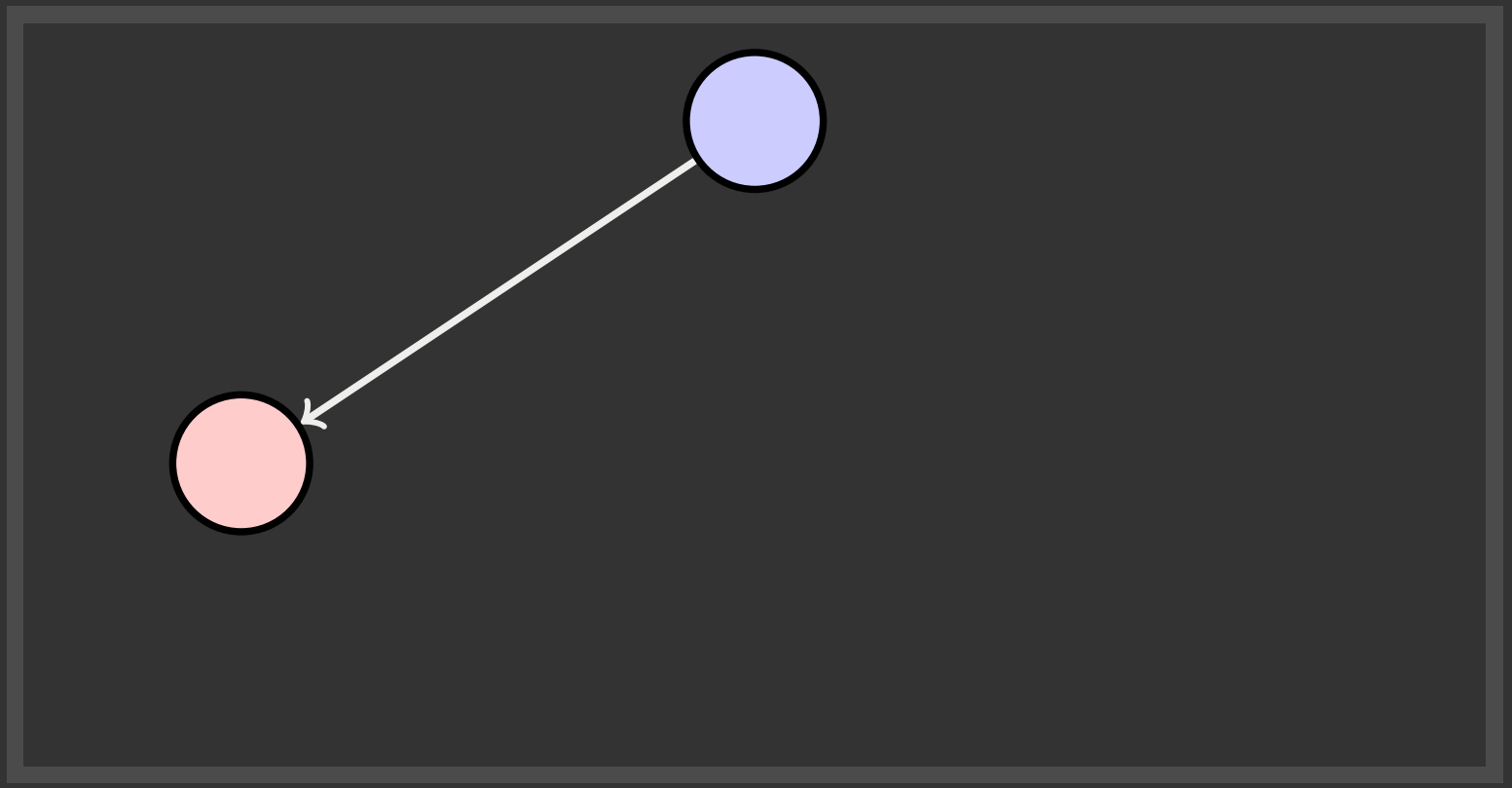
```
Function min-value(state)
    v ← +∞ ;
    for each successor of state do
        v ← min(v, max − value(successor)) ;
        return v;
    end
```

MINIMAX EXAMPLE

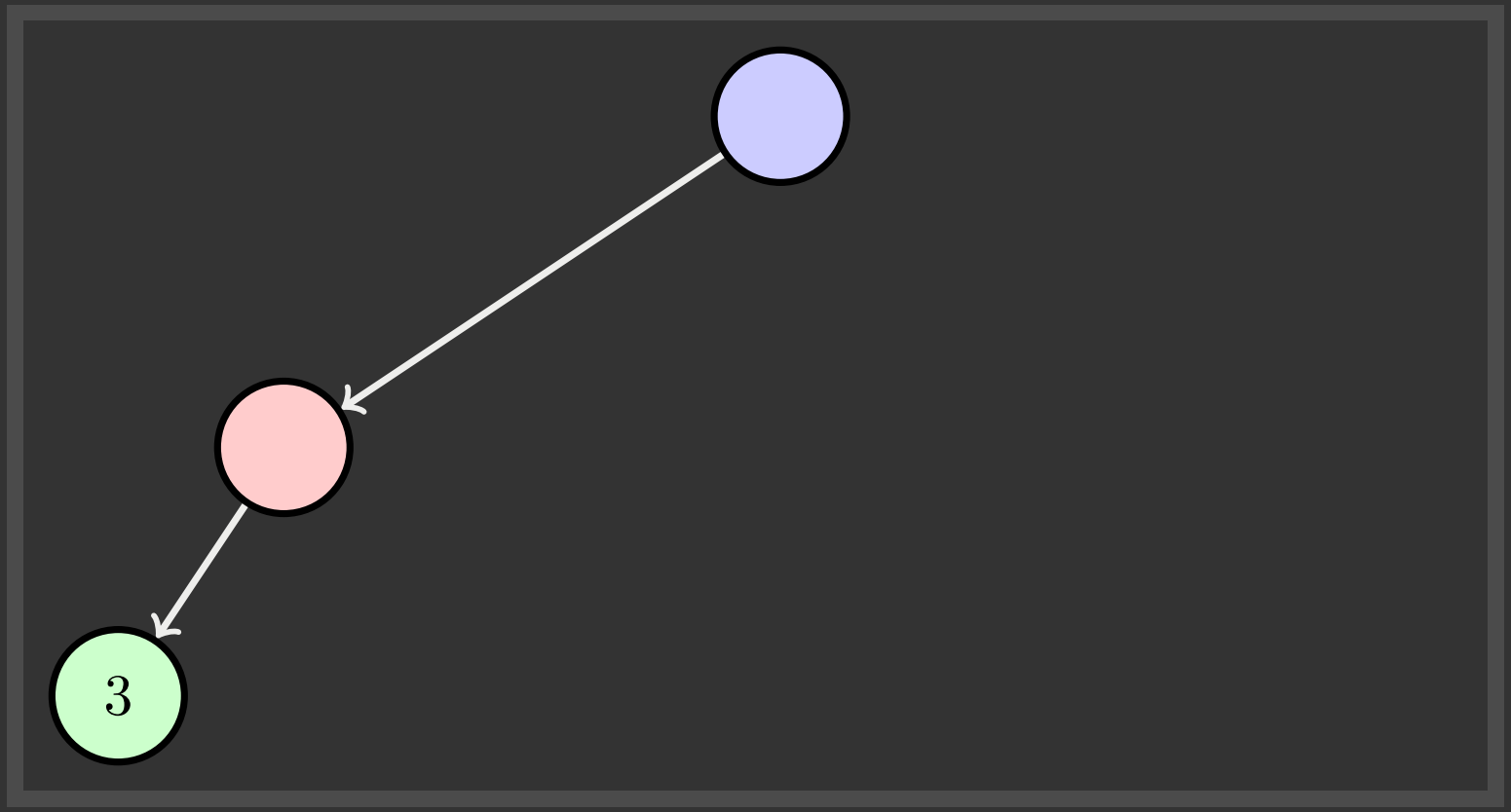
MINIMAX EXAMPLE



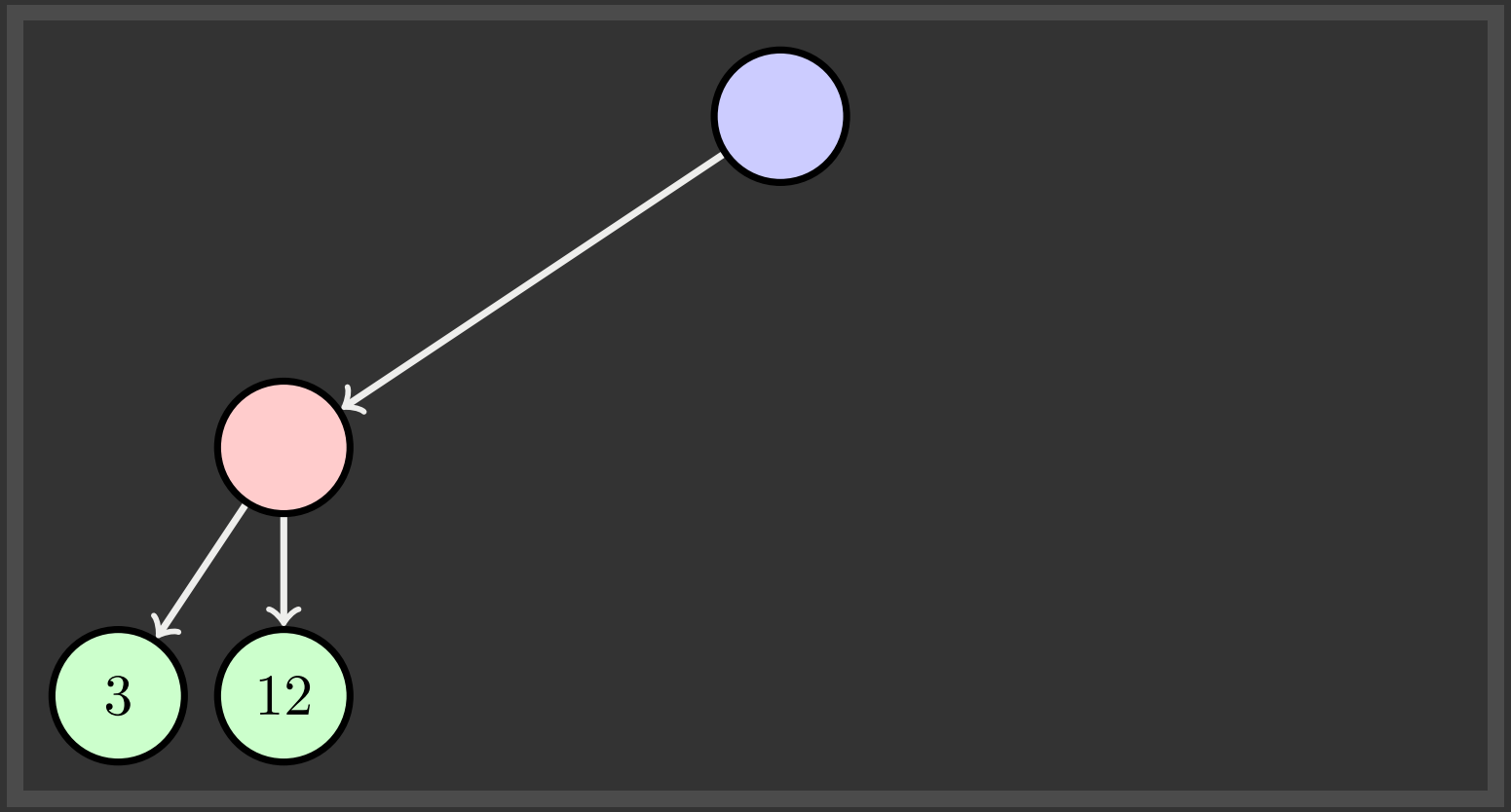
MINIMAX EXAMPLE



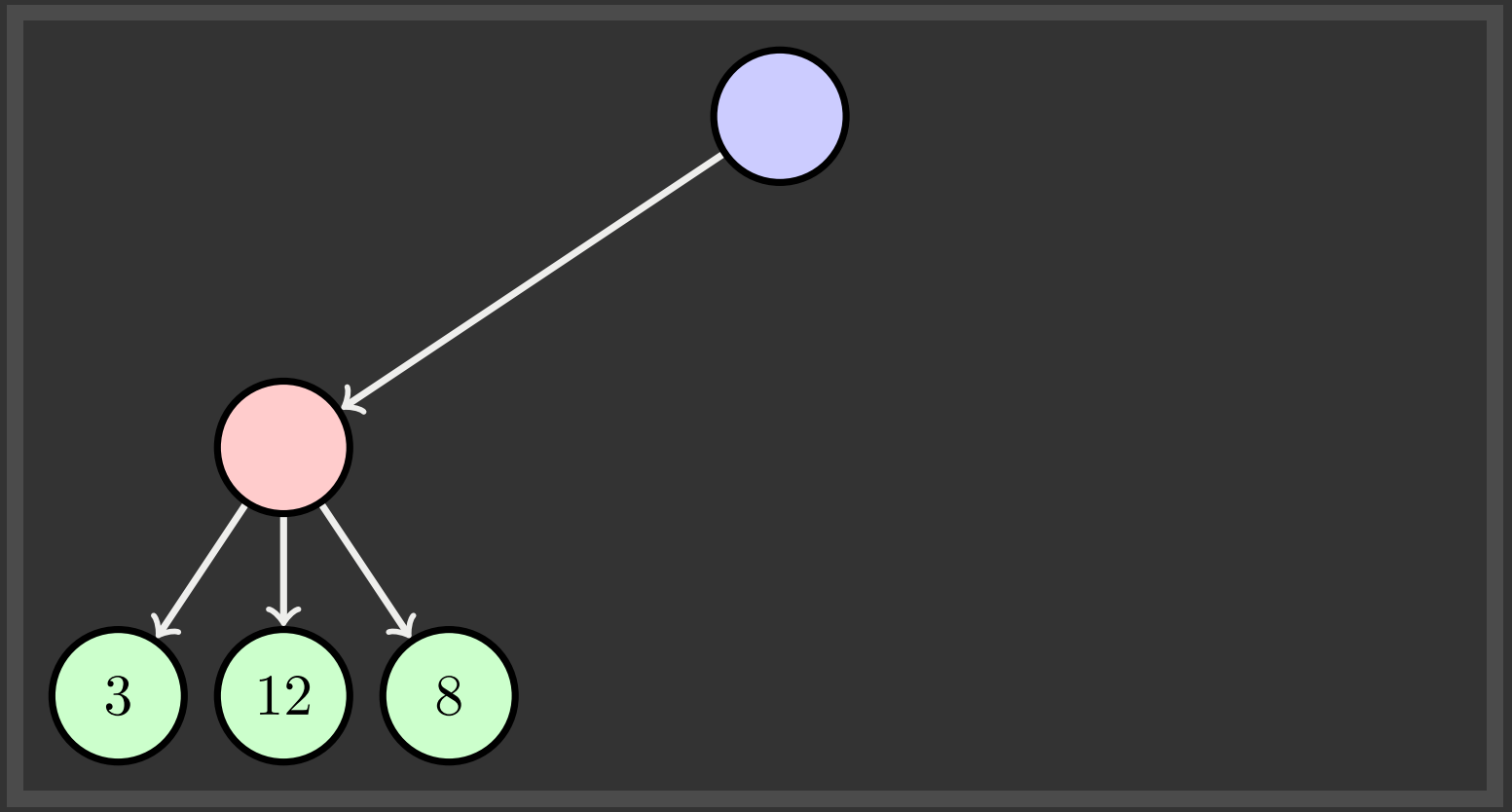
MINIMAX EXAMPLE



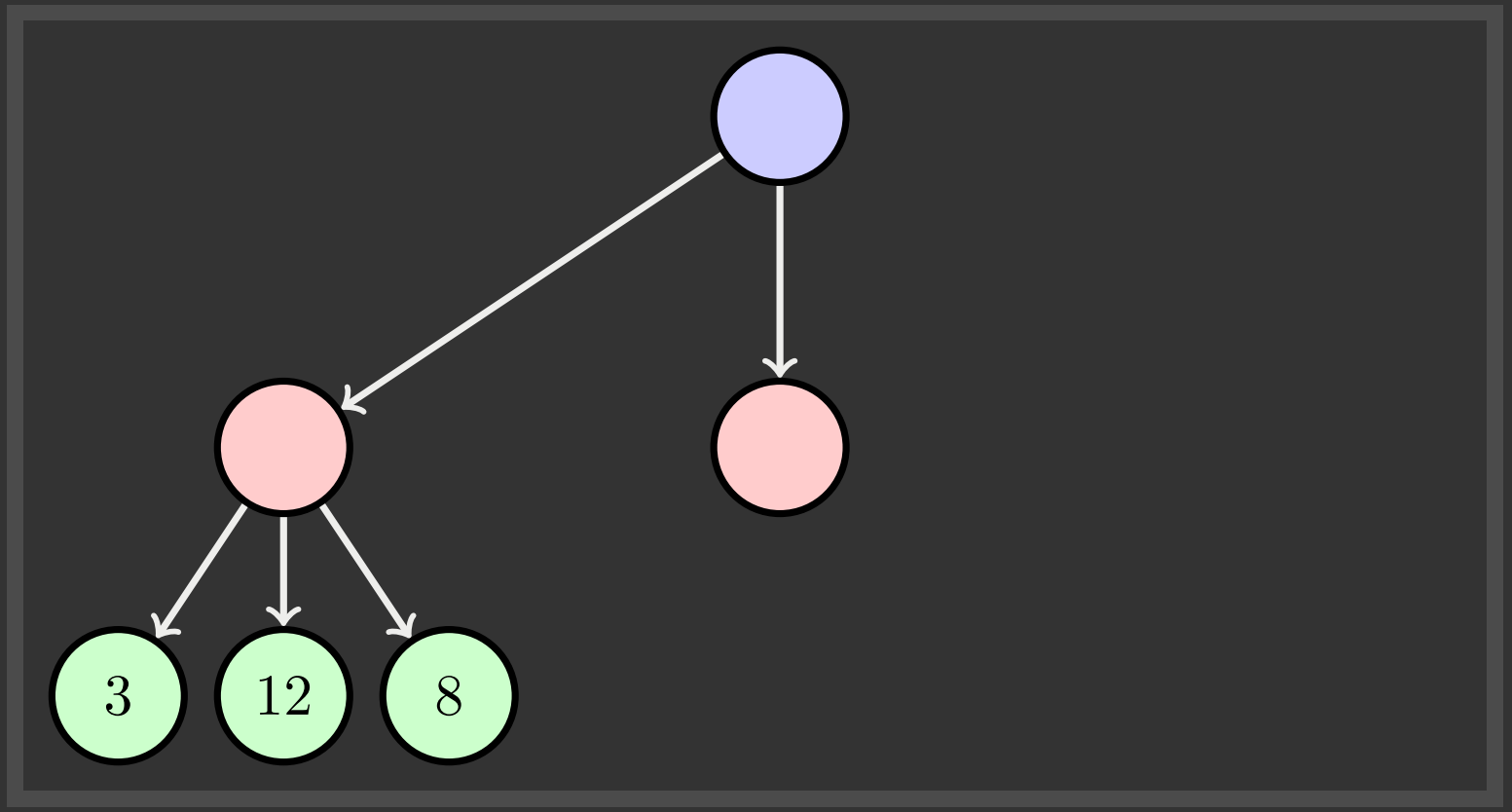
MINIMAX EXAMPLE



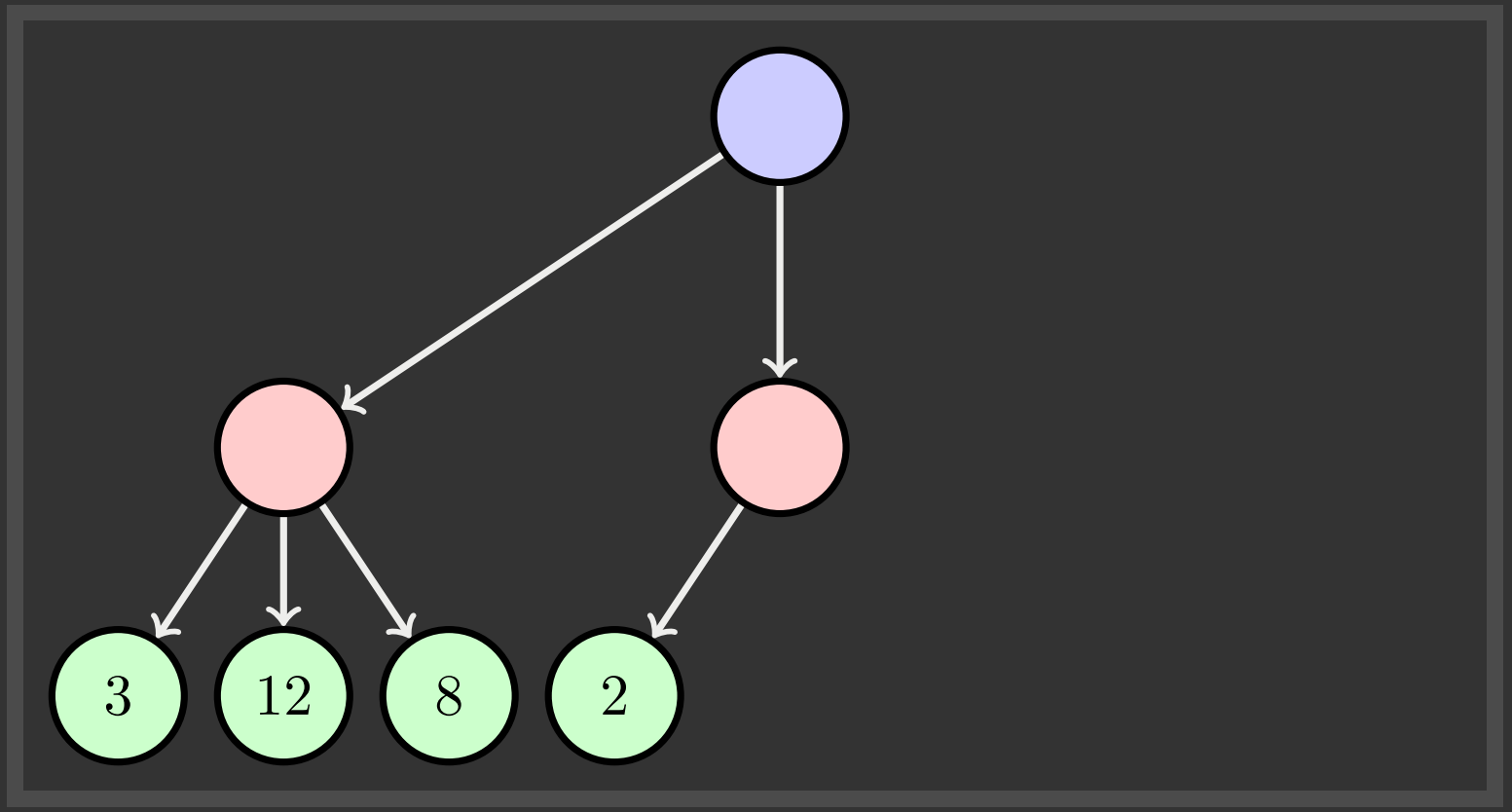
MINIMAX EXAMPLE



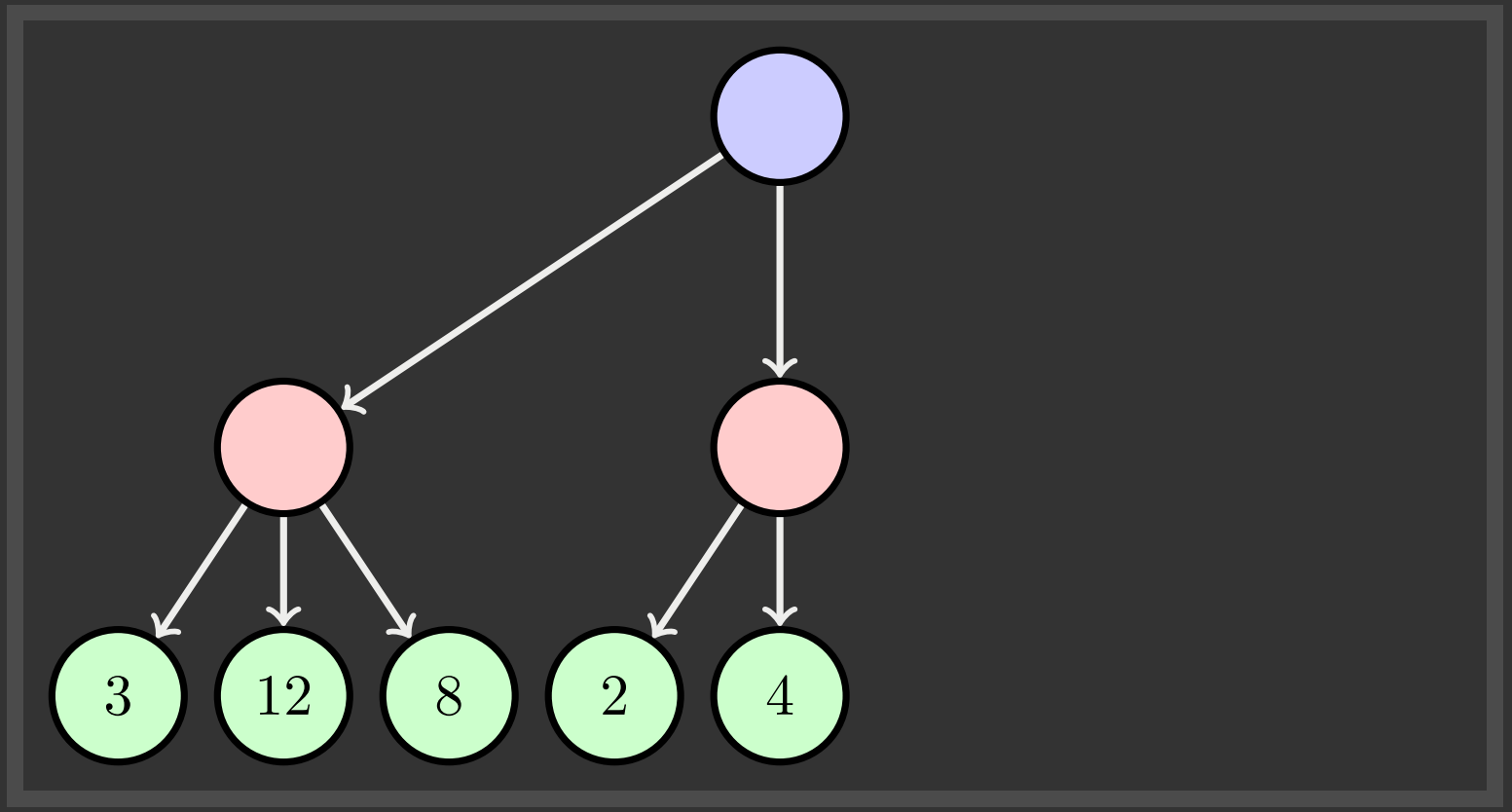
MINIMAX EXAMPLE



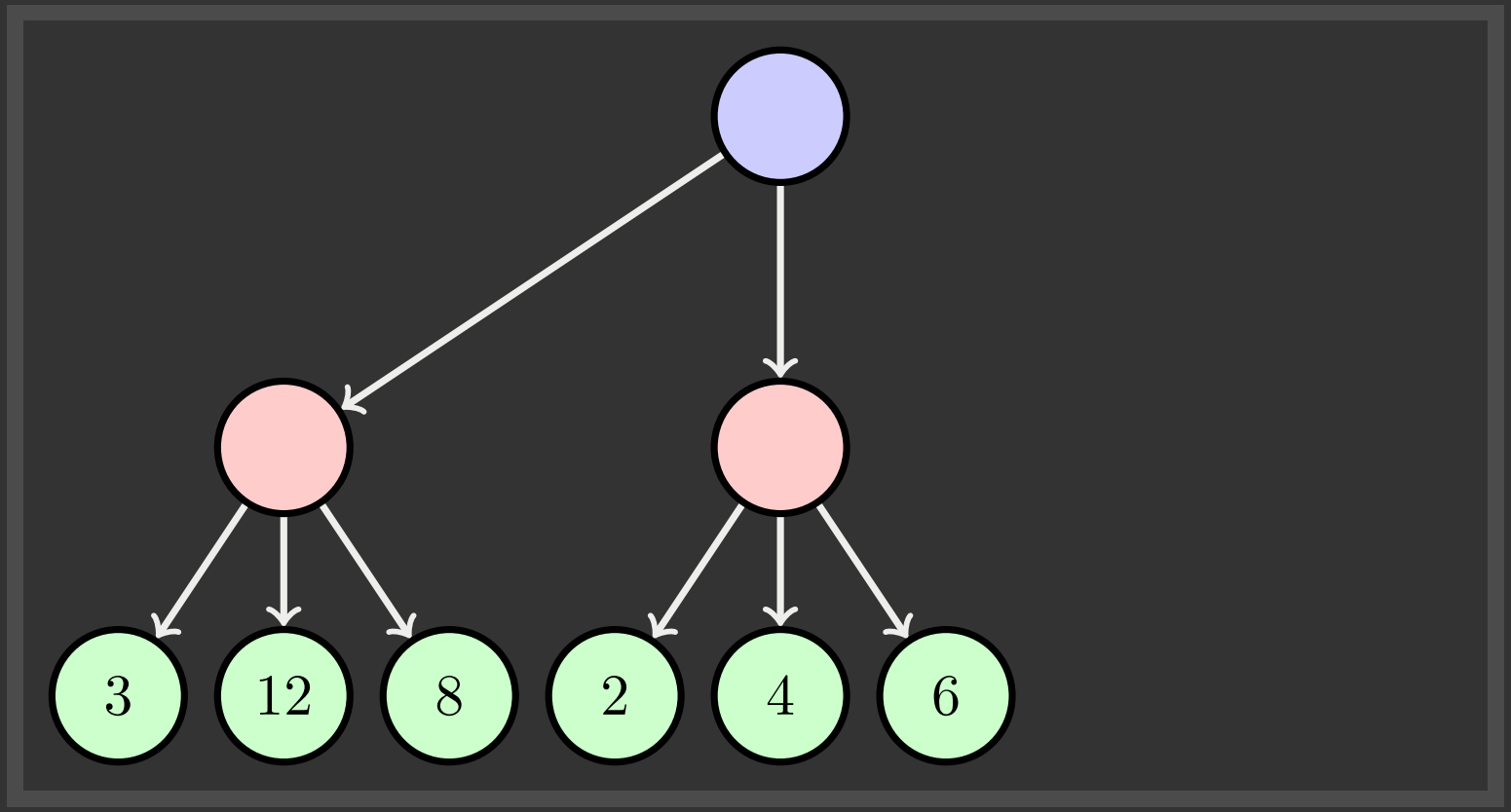
MINIMAX EXAMPLE



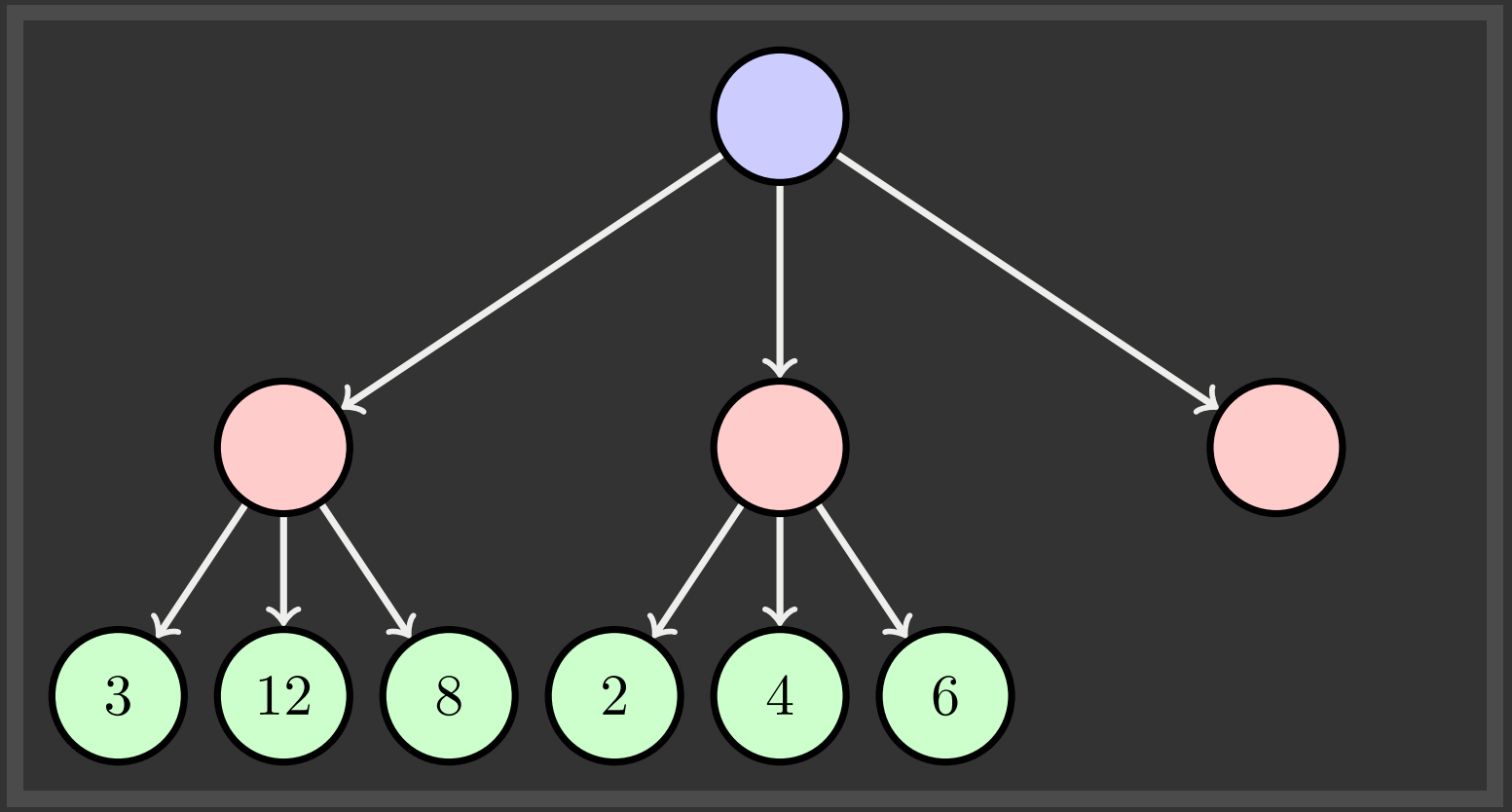
MINIMAX EXAMPLE



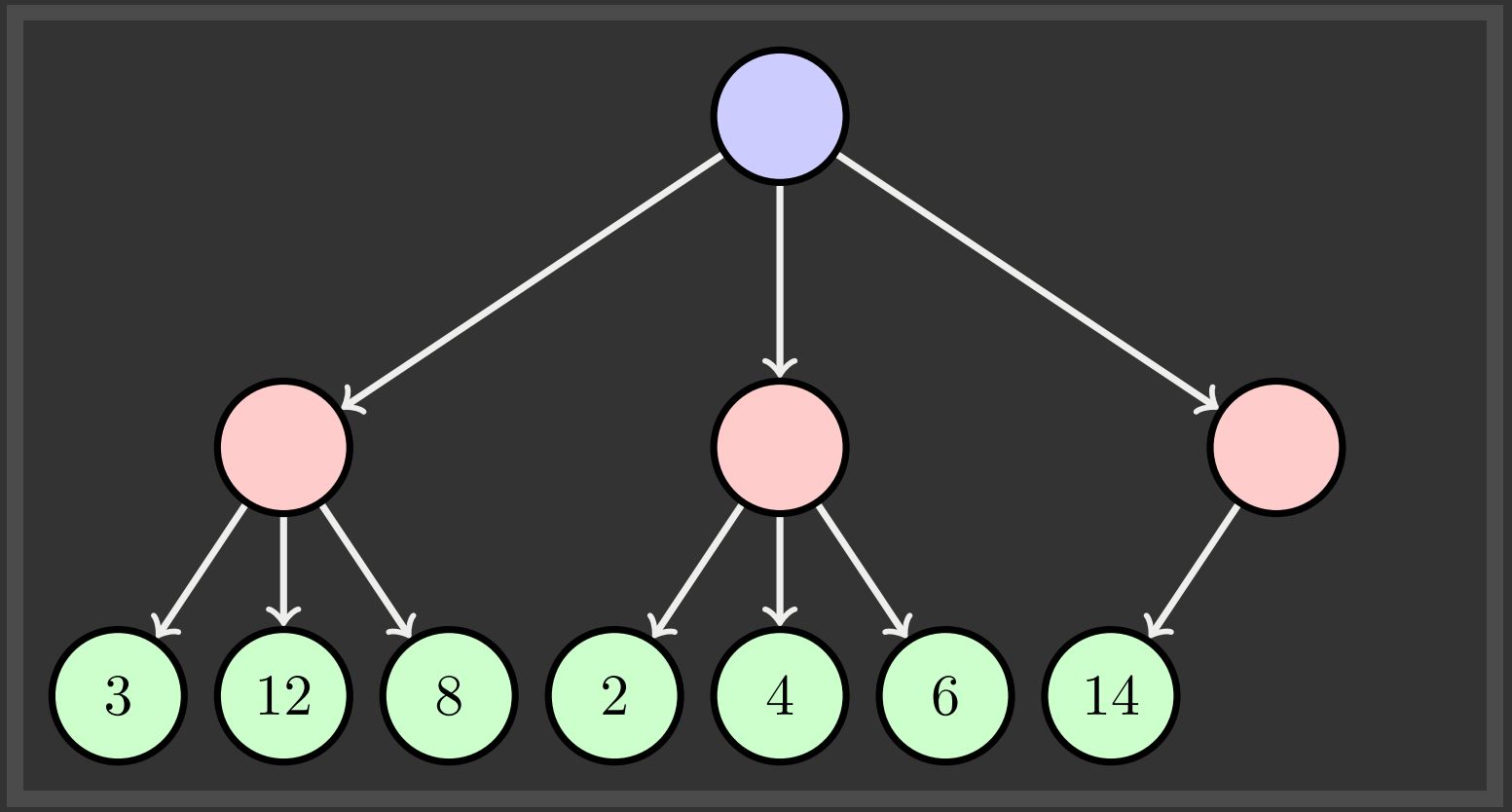
MINIMAX EXAMPLE



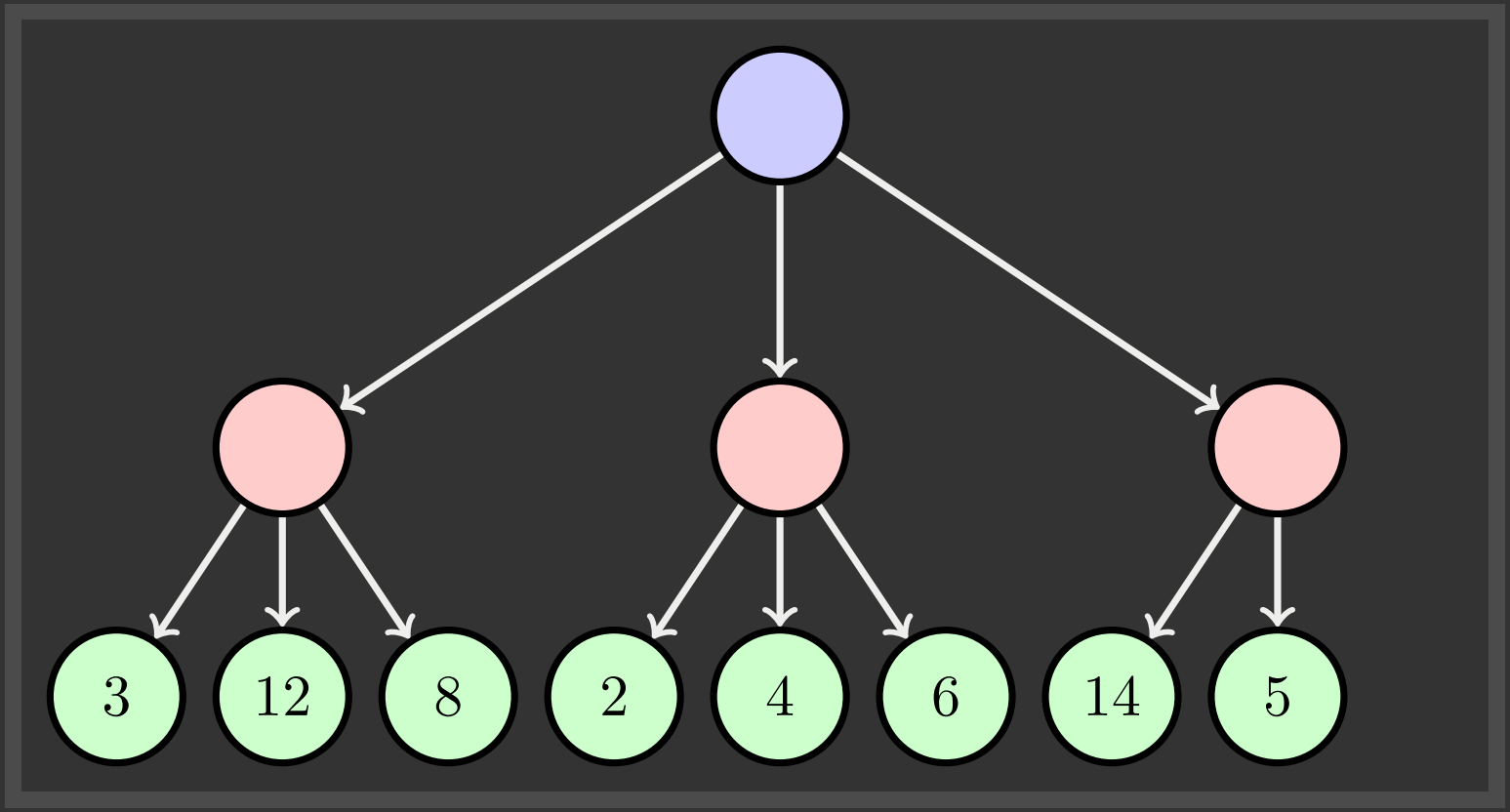
MINIMAX EXAMPLE



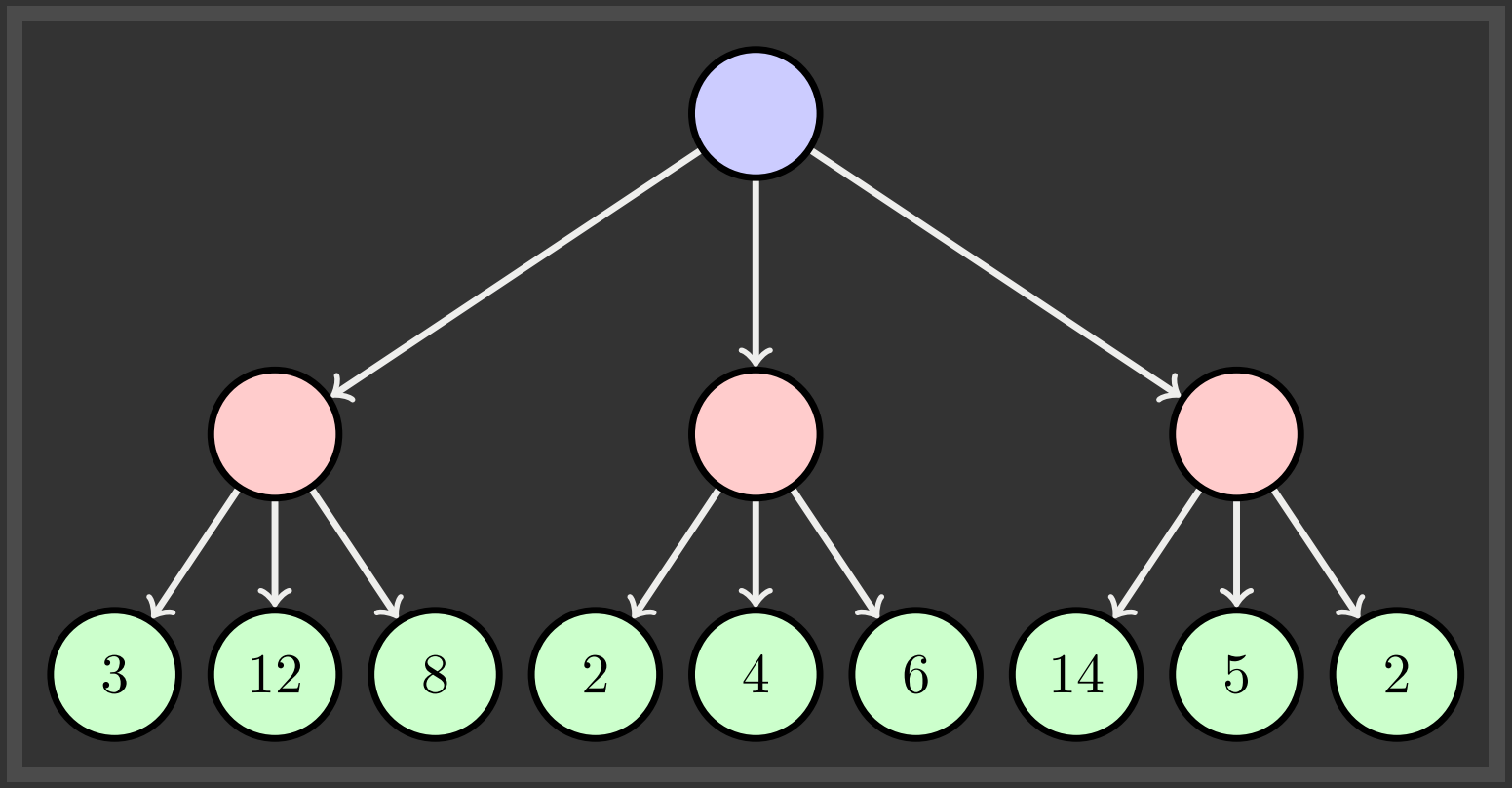
MINIMAX EXAMPLE



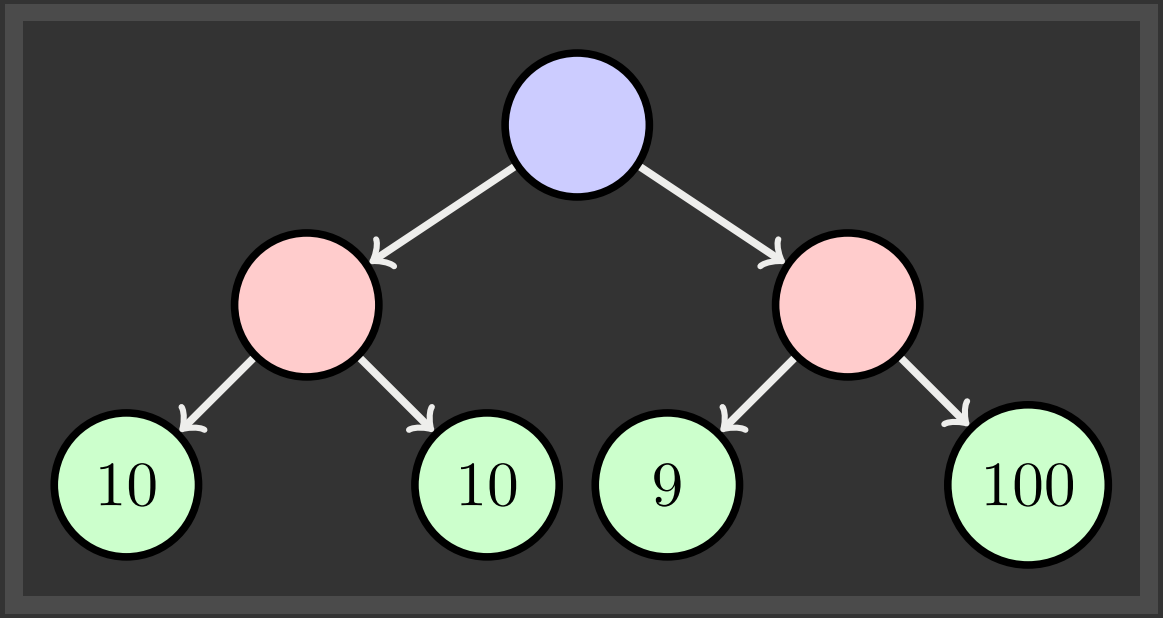
MINIMAX EXAMPLE



MINIMAX EXAMPLE

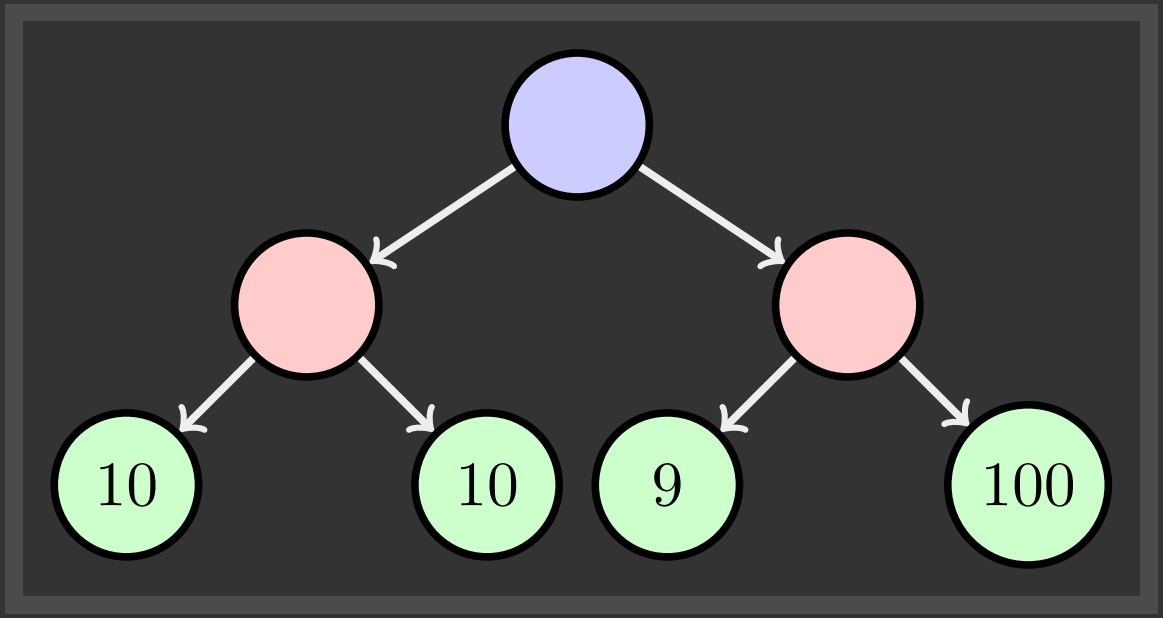


MINIMAX PROPERTIES



Optimal against a perfect player. Otherwise?

MINIMAX PROPERTIES



Optimal against a perfect player. Otherwise?

Minimax value 10, move left. The 100 on the right is bait: it needs the opponent to blunder, and the price of guessing wrong is 9.

MINIMAX EFFICIENCY

How efficient is minimax?



MINIMAX EFFICIENCY

How efficient is minimax?

- Just like (exhaustive) DFS
- Time: $\mathcal{O}(b^m)$
- Space: $\mathcal{O}(bm)$



MINIMAX EFFICIENCY

How efficient is minimax?

- Just like (exhaustive) DFS
- Time: $\mathcal{O}(b^m)$
- Space: $\mathcal{O}(bm)$

Example: For chess, $b \approx 35$, $m \approx 100$



MINIMAX EFFICIENCY

How efficient is minimax?

- Just like (exhaustive) DFS
- Time: $\mathcal{O}(b^m)$
- Space: $\mathcal{O}(bm)$

Example: For chess, $b \approx 35$, $m \approx 100$

- Exact solution is completely infeasible
- But, do we need to explore the whole tree?

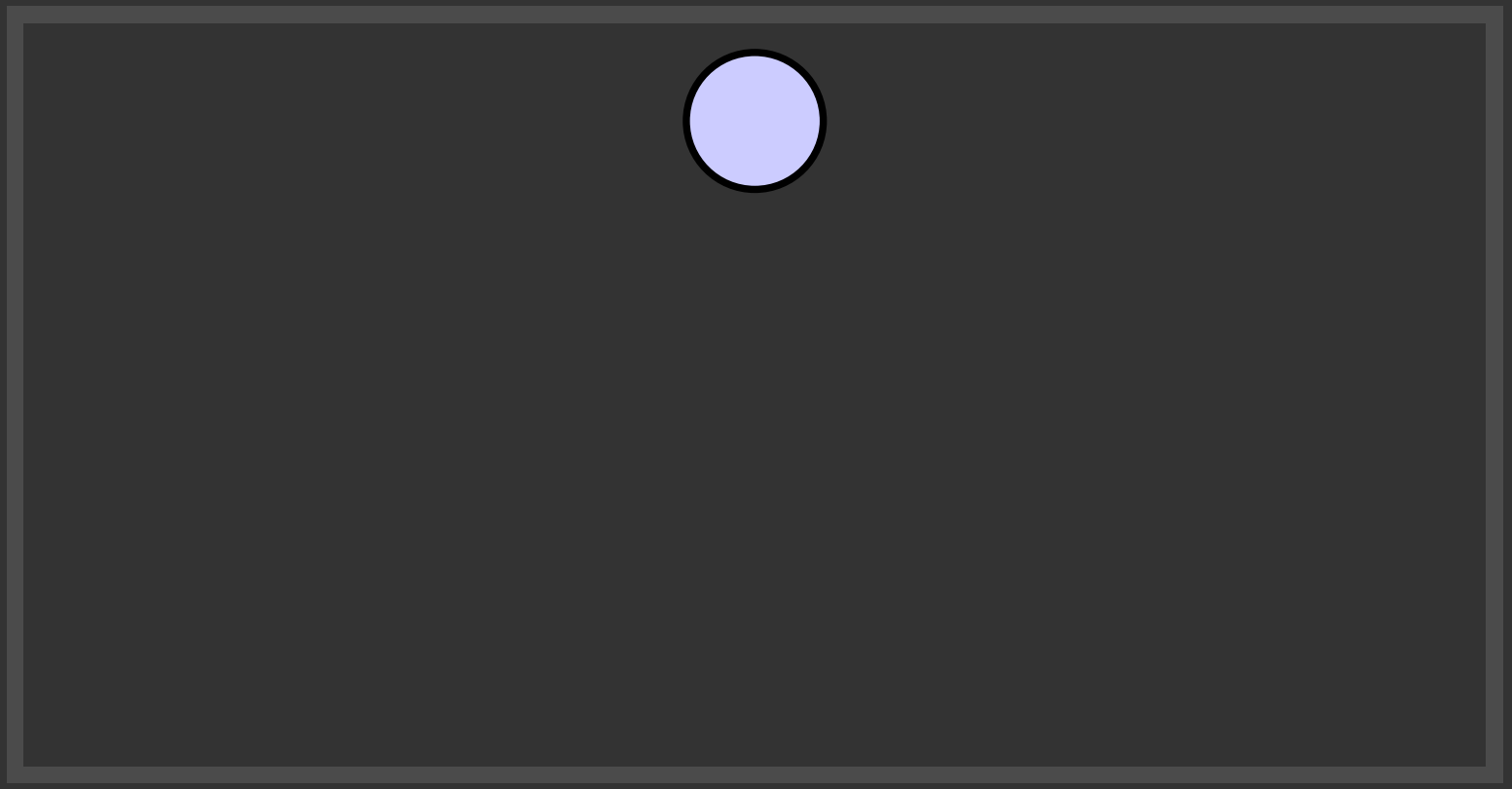


GAME TREE PRUNING

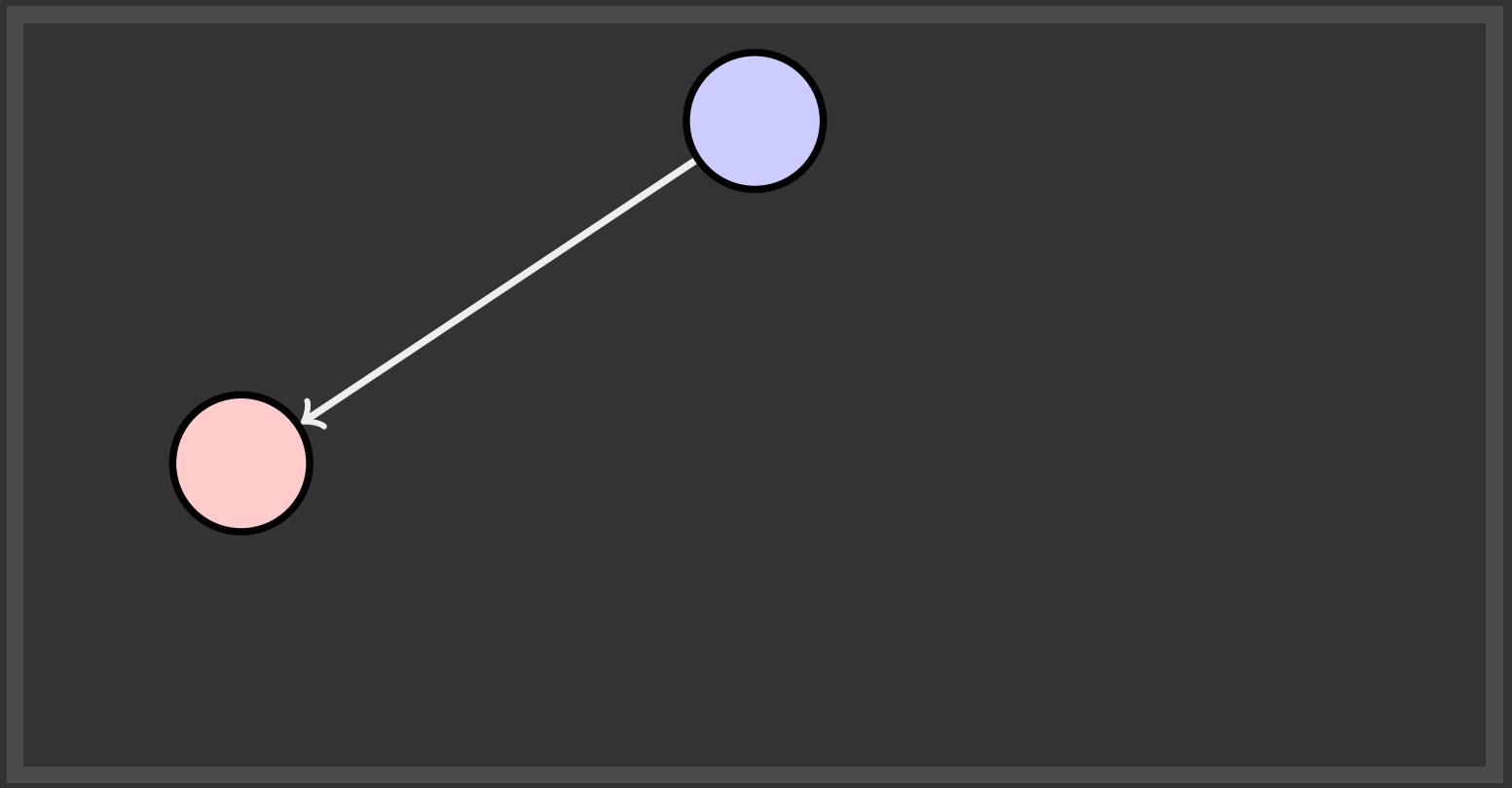


MINIMAX PRUNING

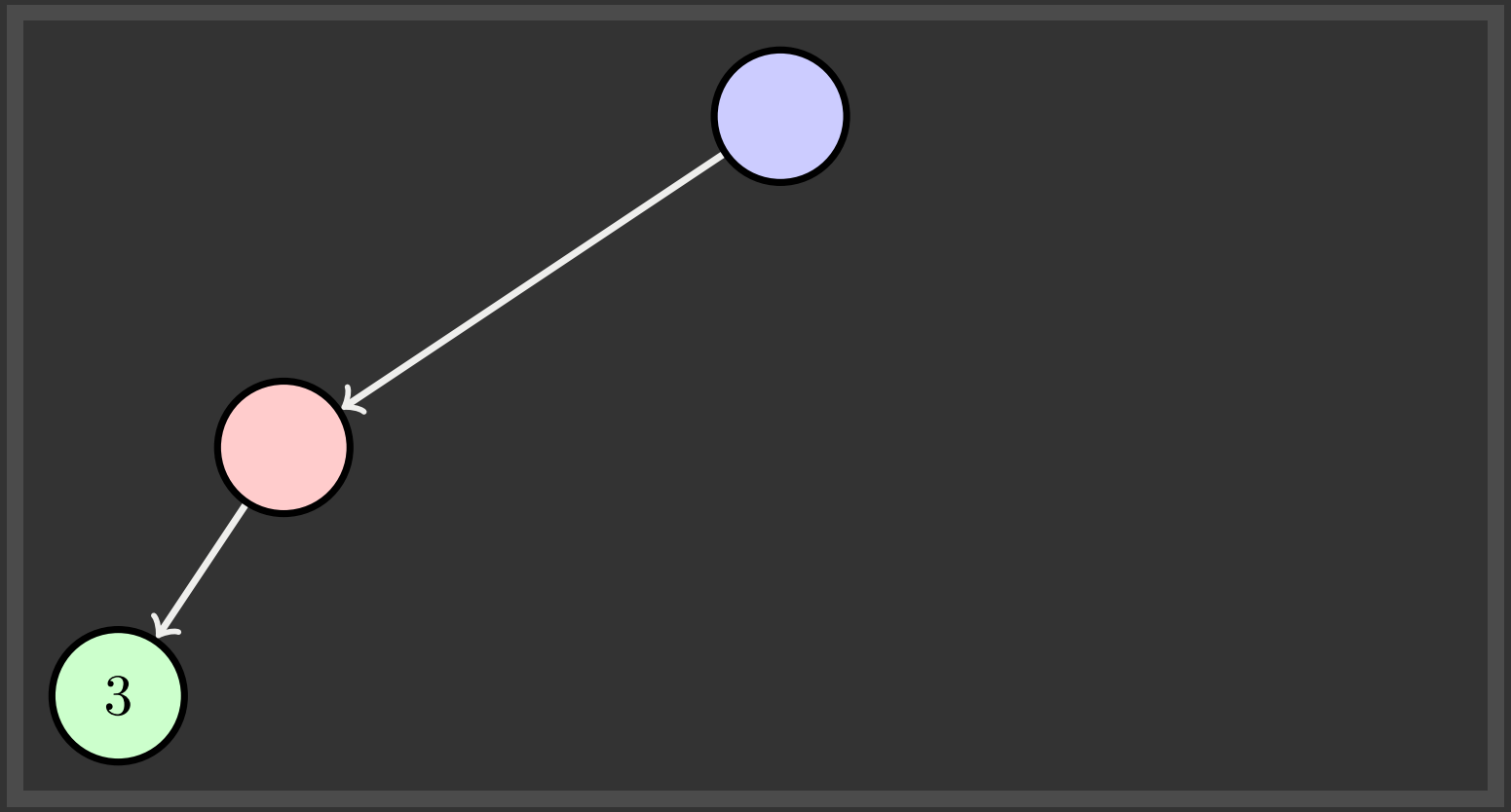
MINIMAX PRUNING



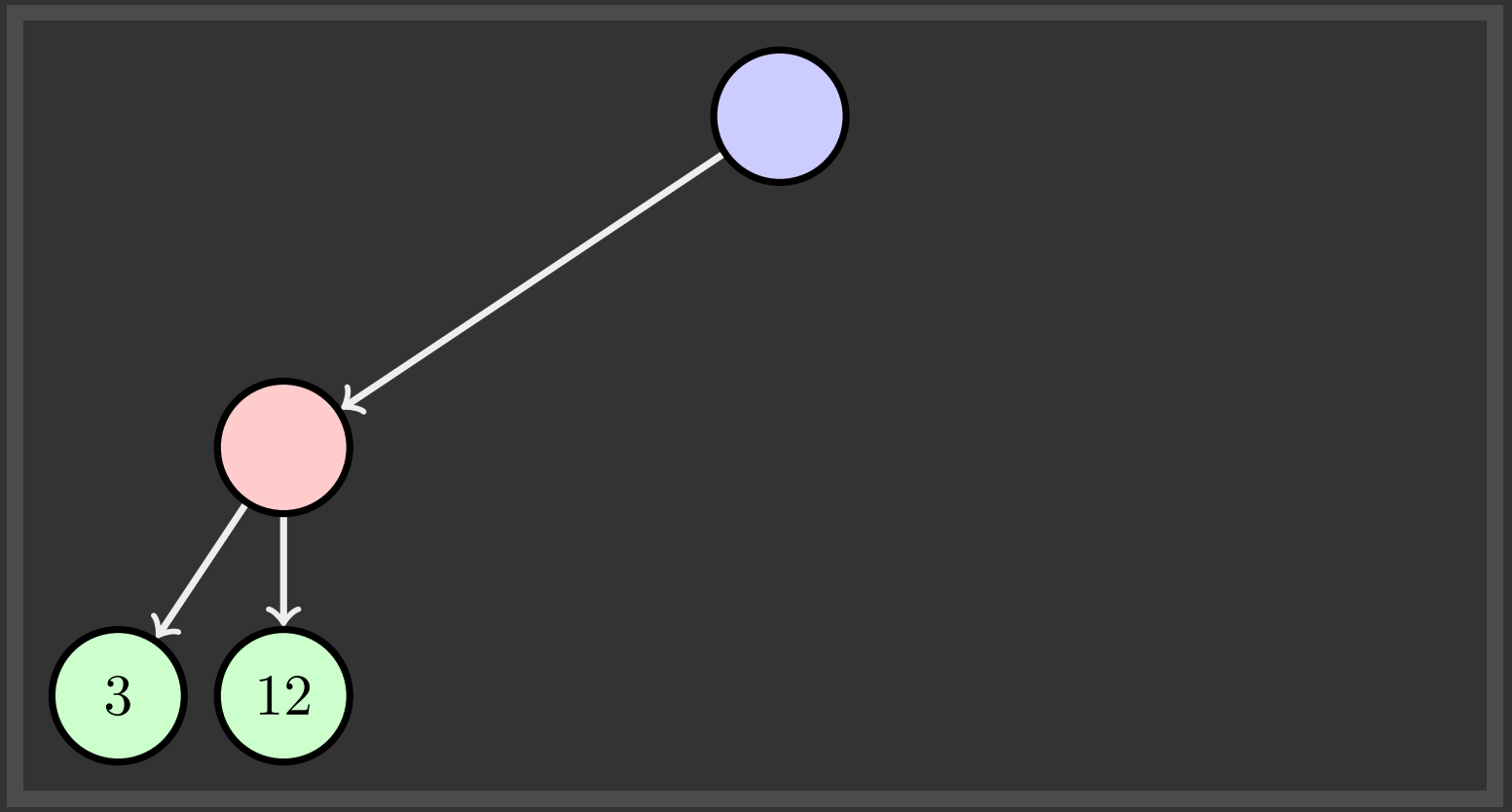
MINIMAX PRUNING



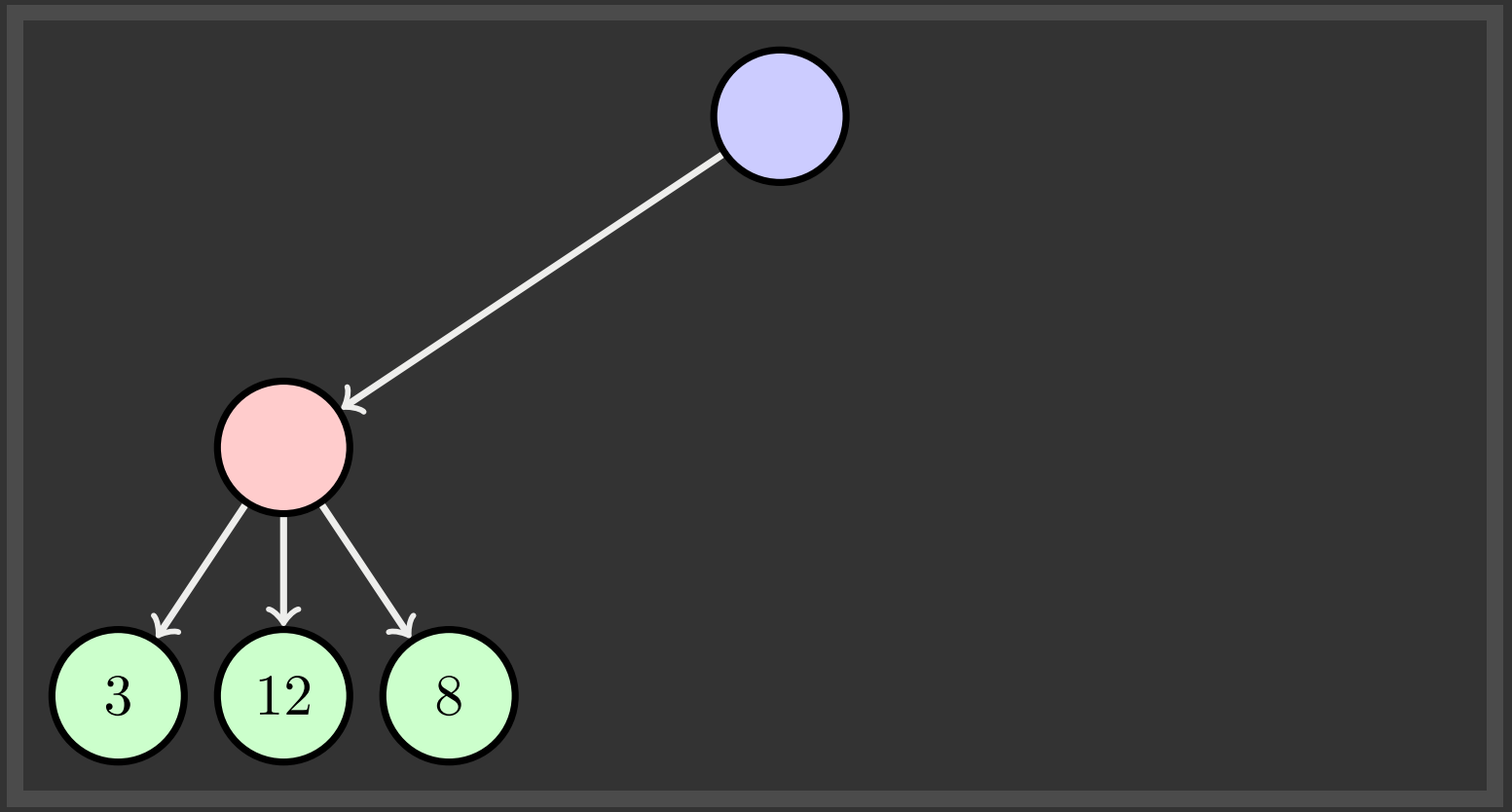
MINIMAX PRUNING



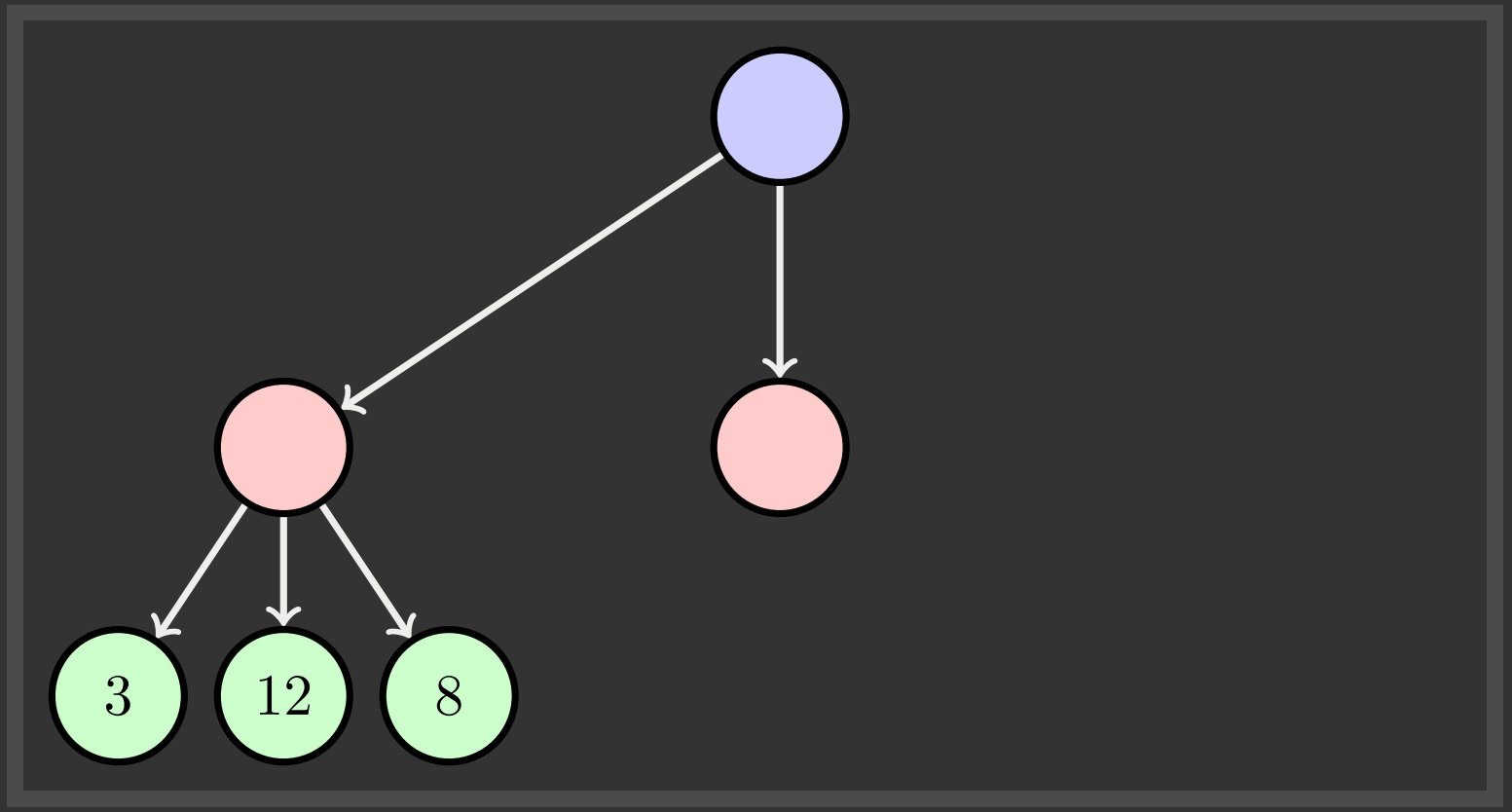
MINIMAX PRUNING



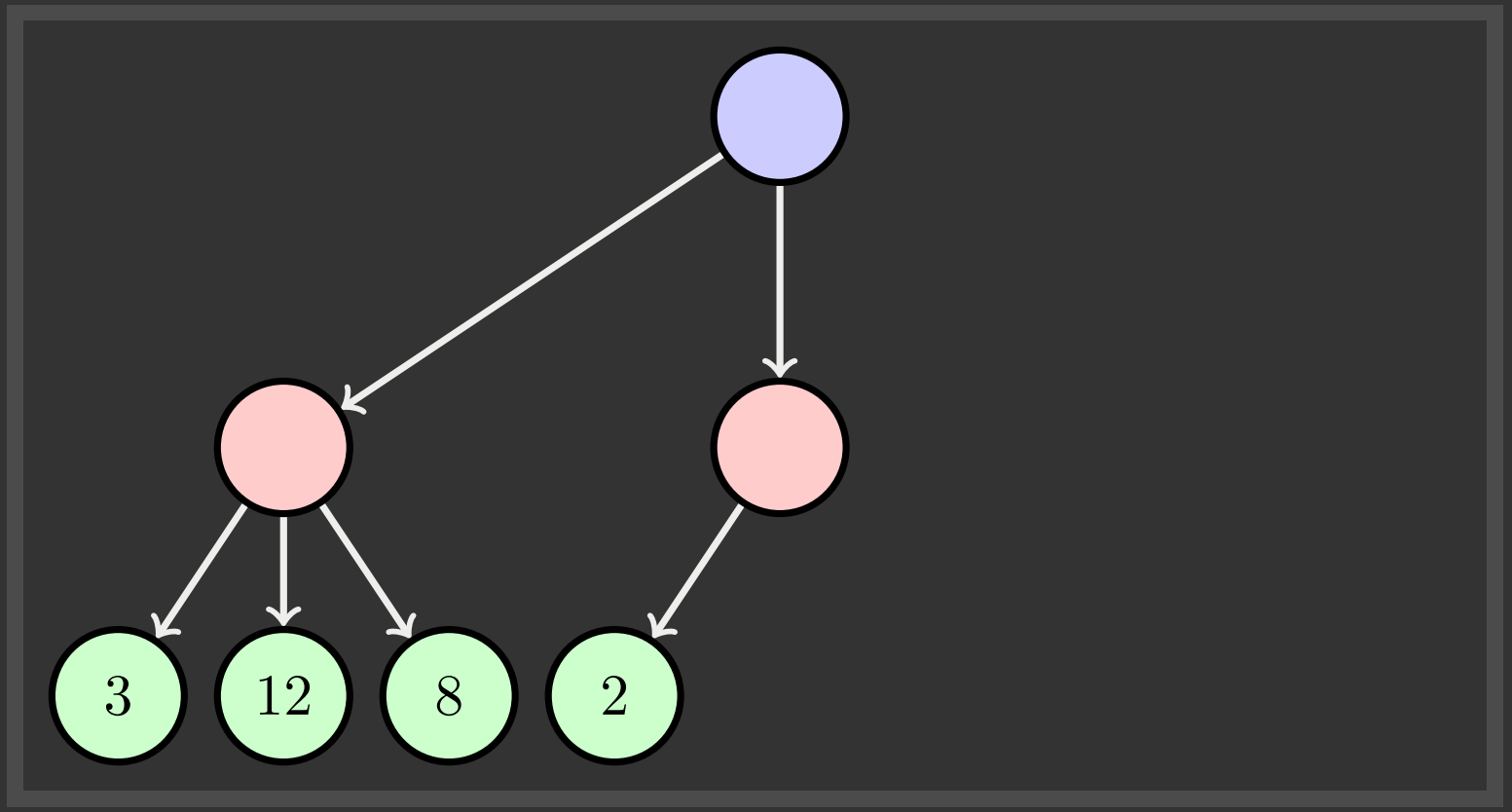
MINIMAX PRUNING



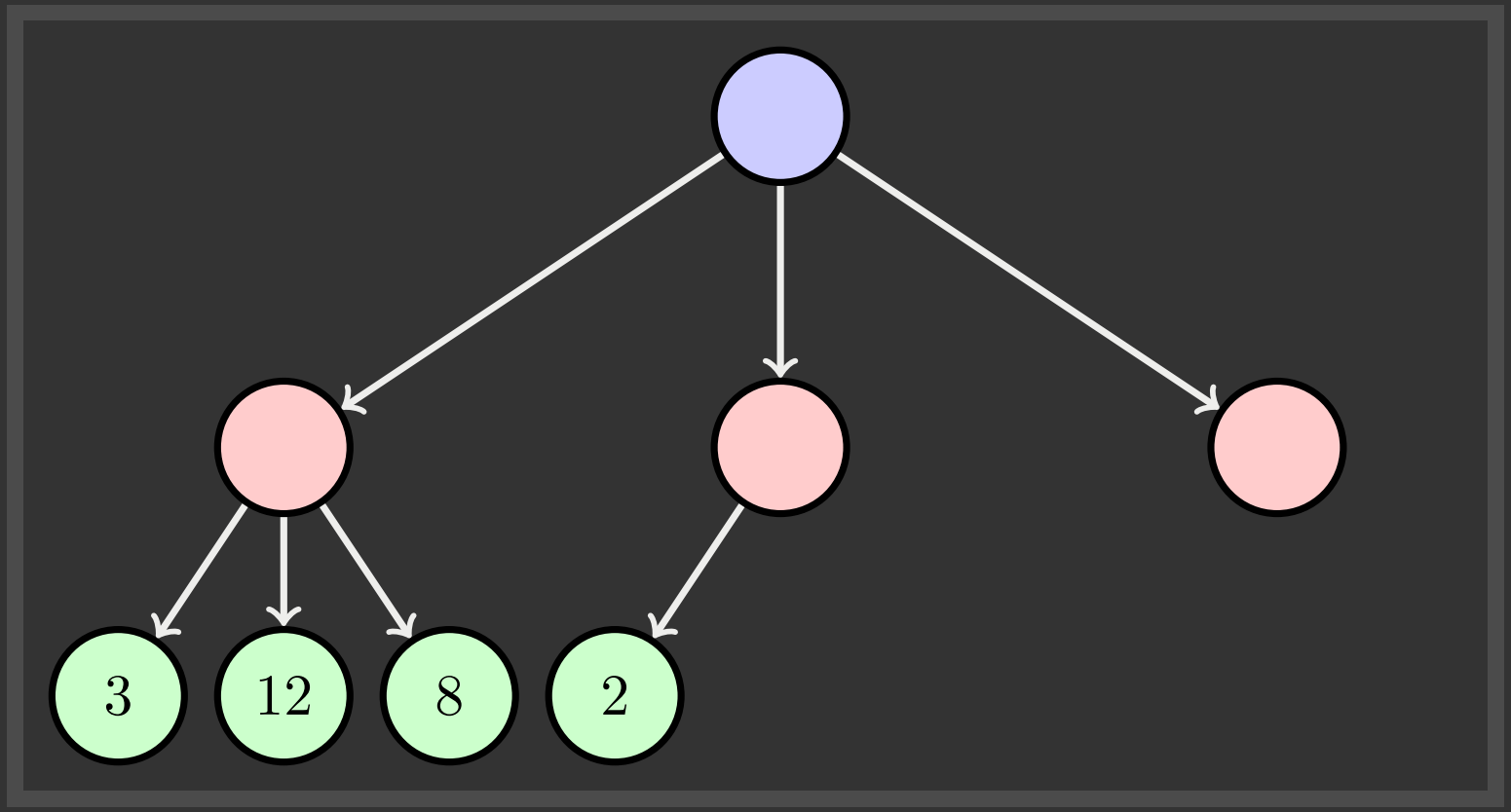
MINIMAX PRUNING



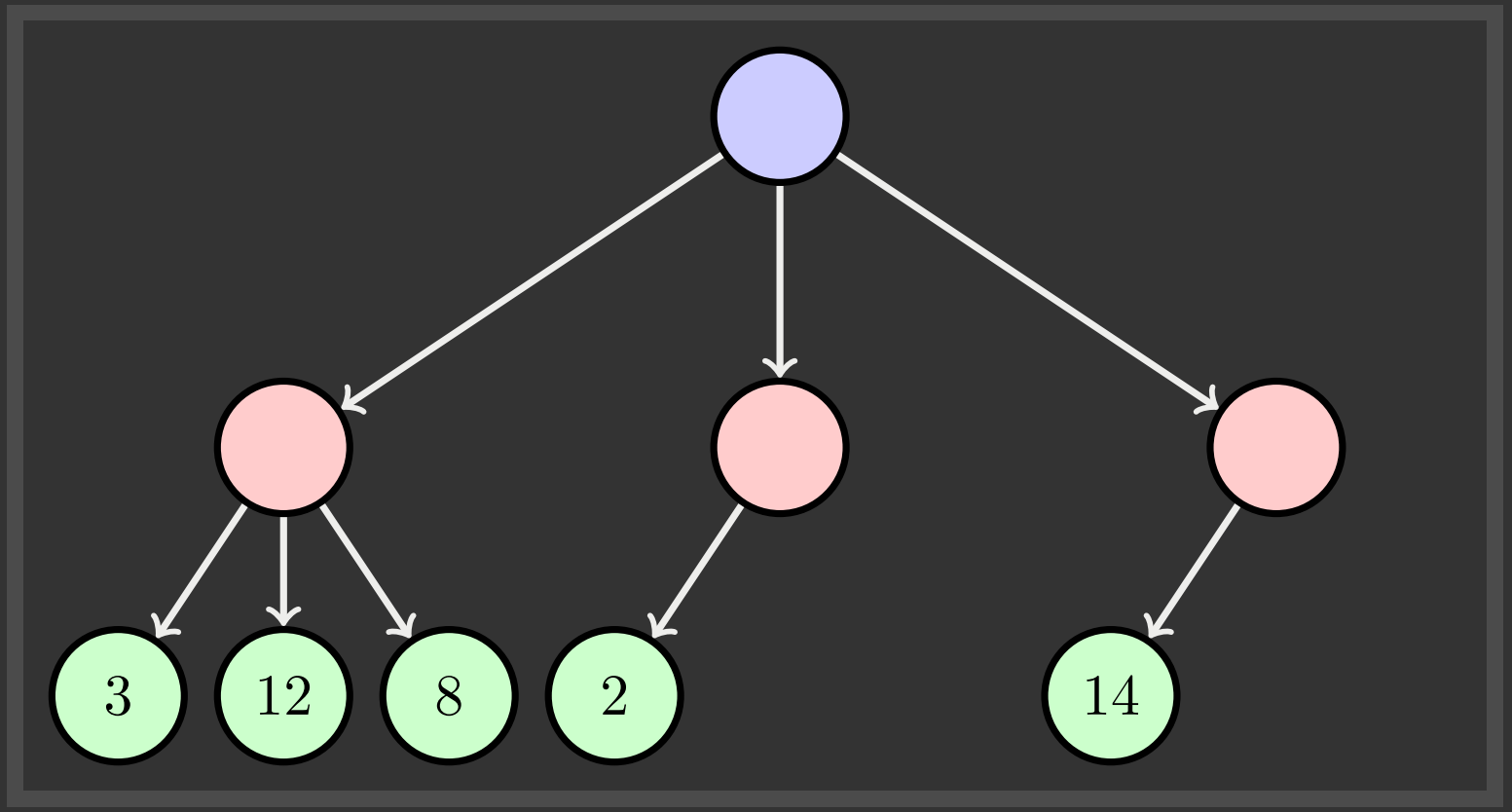
MINIMAX PRUNING



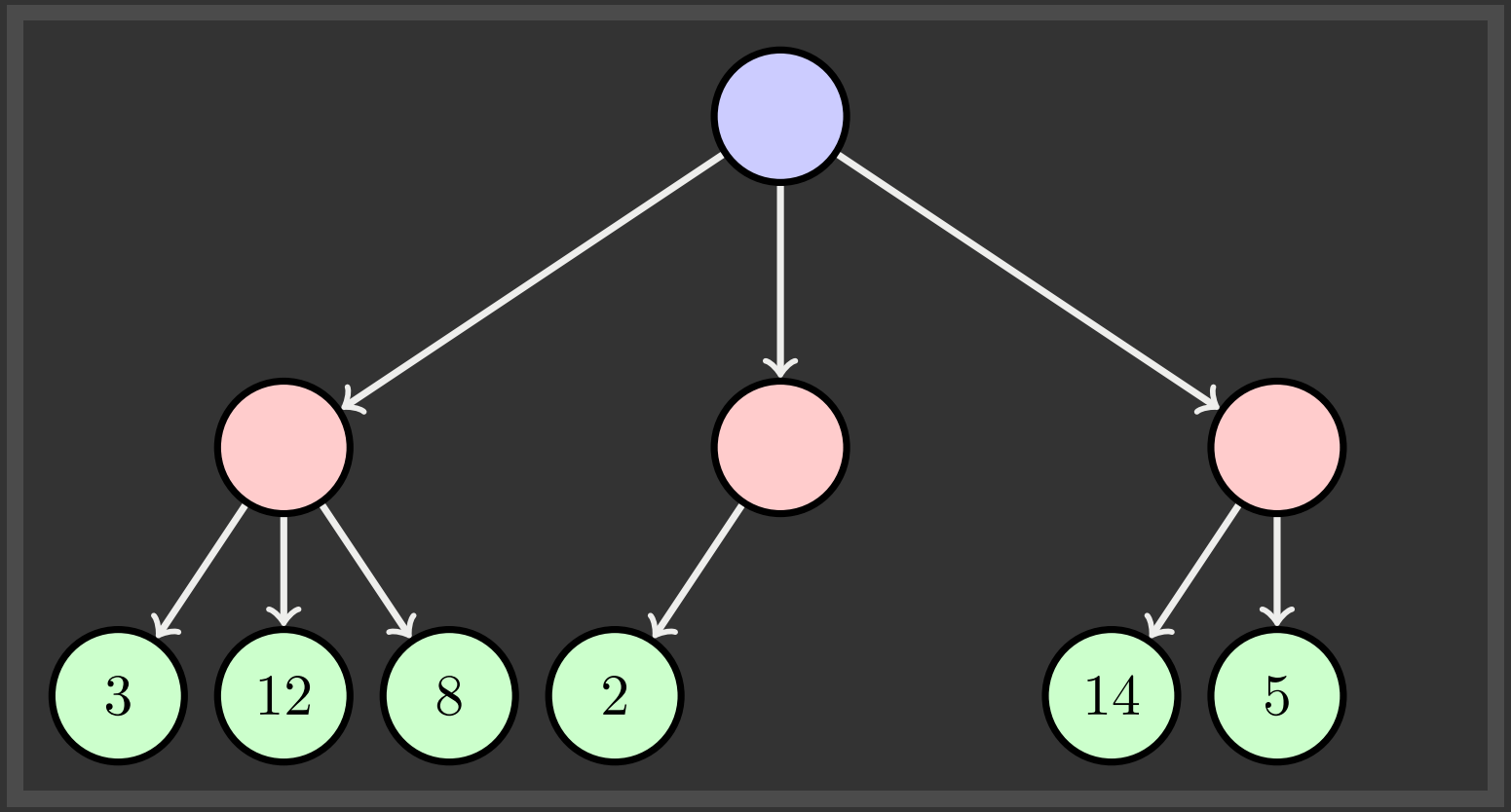
MINIMAX PRUNING



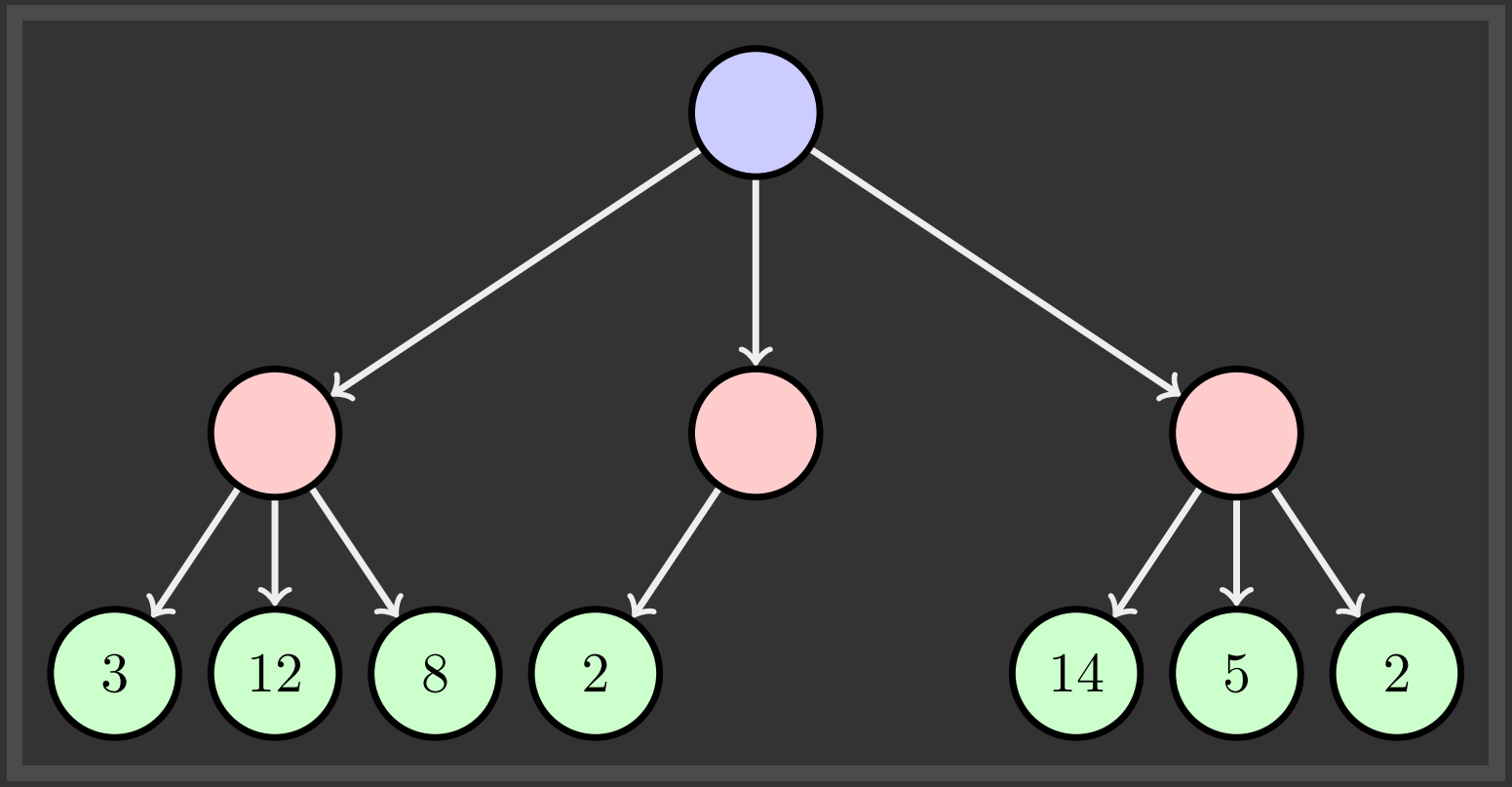
MINIMAX PRUNING



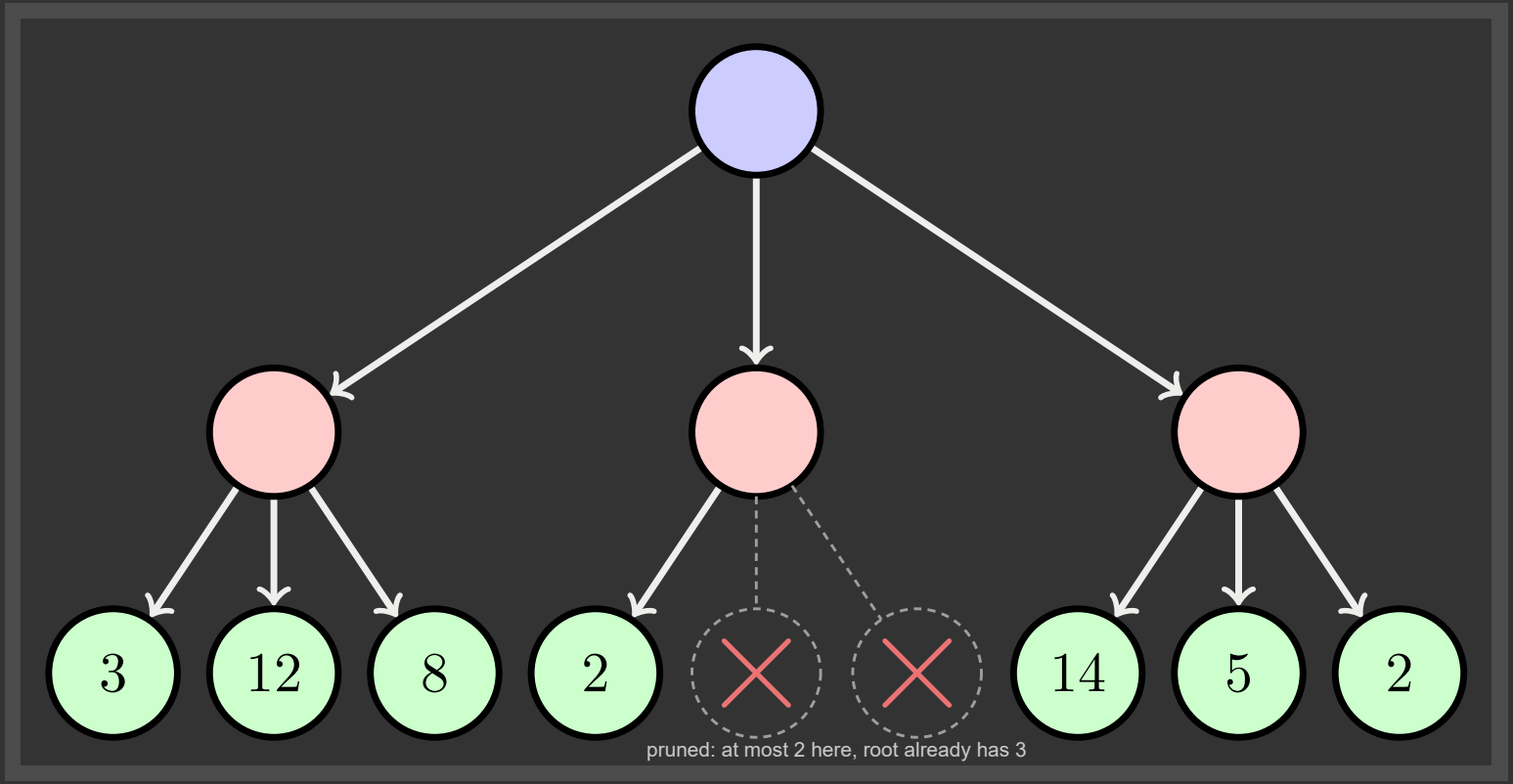
MINIMAX PRUNING



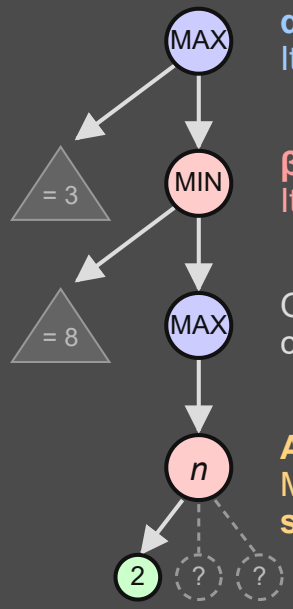
MINIMAX PRUNING



MINIMAX PRUNING



A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

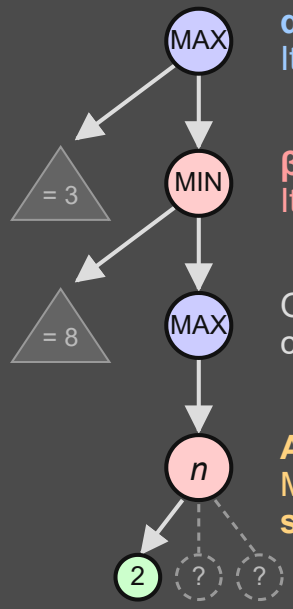
β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

Only values in (α, β) = (3, 8) can still change the decision at the root.

At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

While searching, carry two numbers for the current path from the root:

A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

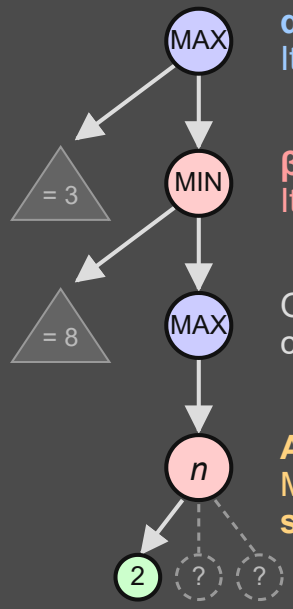
Only values in (α, β) = (3, 8) can still change the decision at the root.

At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

While searching, carry two numbers for the current path from the root:

- α: the best value MAX is already guaranteed at a choice point above: a **floor**

A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

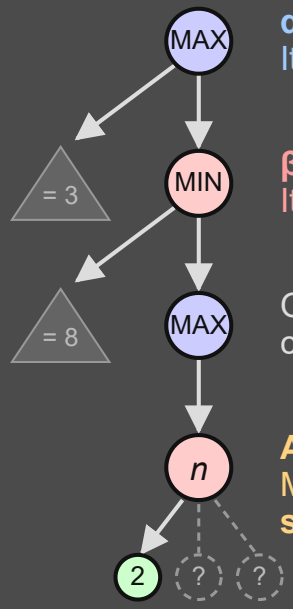
Only values in (α, β) = (3, 8) can still change the decision at the root.

At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

While searching, carry two numbers for the current path from the root:

- α: the best value MAX is already guaranteed at a choice point above: a **floor**
- β: the best value MIN is already guaranteed above: a **ceiling**

A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

Only values in (α, β) = (3, 8) can still change the decision at the root.

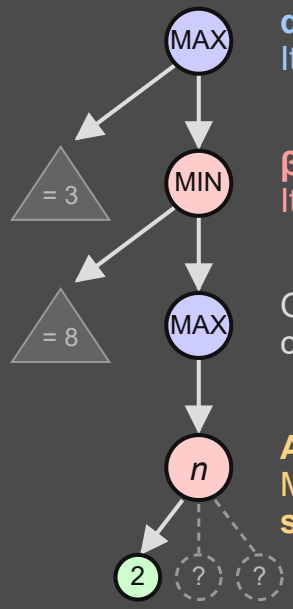
At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

While searching, carry two numbers for the current path from the root:

- α: the best value MAX is already guaranteed at a choice point above: a **floor**
- β: the best value MIN is already guaranteed above: a **ceiling**

Only values strictly between α and β can still change the decision at the root

A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

Only values in (α, β) = (3, 8) can still change the decision at the root.

At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

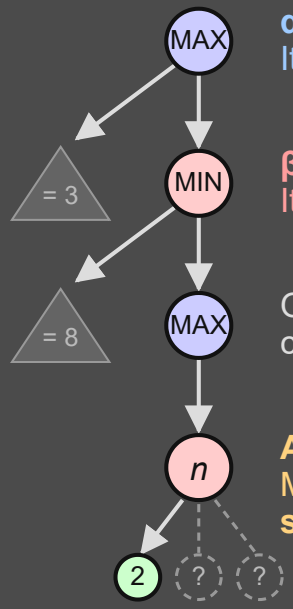
While searching, carry two numbers for the current path from the root:

- α: the best value MAX is already guaranteed at a choice point above: a **floor**
- β: the best value MIN is already guaranteed above: a **ceiling**

Only values strictly between α and β can still change the decision at the root

At a MIN node n, v only goes down. Once $v \leq \alpha$, MAX above will never choose n: stop expanding n

A-B PRUNING: THE IDEA



α = 3: MAX already has 3 here.
It will never accept less on this path.

β = 8: MIN can already hold MAX to 8.
It will never allow more on this path.

Only values in (α, β) = (3, 8) can still change the decision at the root.

At n: first child is 2, so v ≤ 2 ≤ α.
MAX above will never choose n:
stop, skip the other children.

While searching, carry two numbers for the current path from the root:

- α: the best value MAX is already guaranteed at a choice point above: a **floor**
- β: the best value MIN is already guaranteed above: a **ceiling**

Only values strictly between α and β can still change the decision at the root

At a MIN node n, v only goes down. Once $v \leq \alpha$, MAX above will never choose n: stop expanding n

At a MAX node, v only goes up. Once $v \geq \beta$, MIN above will never allow it: stop

A-B IMPLEMENTATION

Dispatch:

α: MAX's best option on path to root

β: MIN's best option on path to root

MAX Agent:

```
def max-value(state, α, β):
    initialize v = −∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

MIN Agent:

```
def min-value(state, α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```



A-B IMPLEMENTATION

Dispatch:

α: MAX's best option on path to root

β: MIN's best option on path to root

MAX Agent:

```
def max-value(state, α, β):
    initialize v = −∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

MIN Agent:

```
def min-value(state, α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

Pass down: the root calls max-value(state, $-\infty$, $+\infty$): no floor, no ceiling yet; every recursive call hands its current (α, β) to its children

A-B IMPLEMENTATION

Dispatch:

α: MAX's best option on path to root

β: MIN's best option on path to root

MAX Agent:

```
def max-value(state, α, β):
    initialize v = −∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

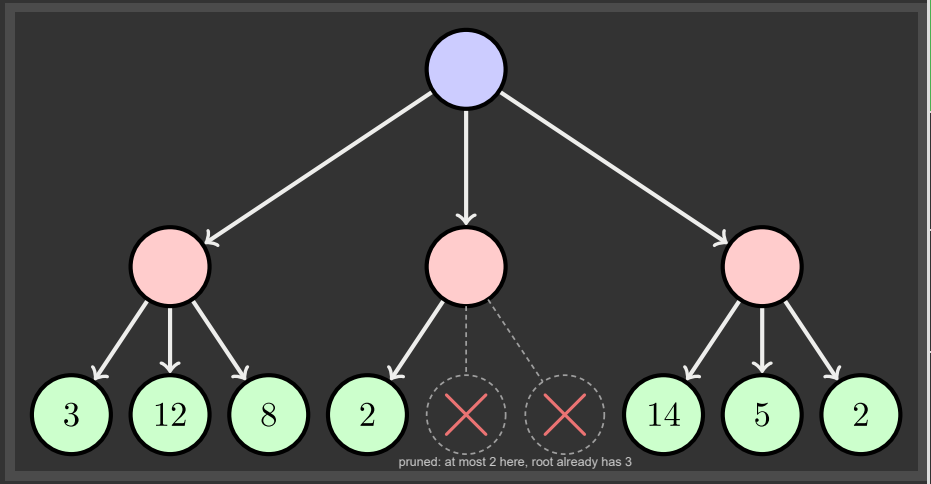
MIN Agent:

```
def min-value(state, α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

Pass down: the root calls max-value(state, $-\infty$, $+\infty$): no floor, no ceiling yet; every recursive call hands its current (α, β) to its children

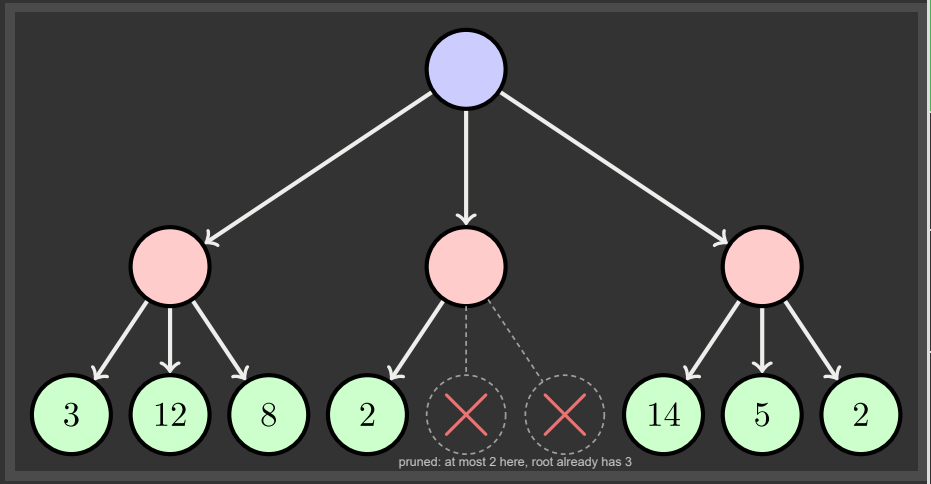
Test, then update: MAX: after each child, if $v \geq \beta$ return v (MIN above will not allow it), else $\alpha \leftarrow \mathbf{max}(\alpha, v)$. MIN: if $v \leq \alpha$ return v, else $\beta \leftarrow \mathbf{min}(\beta, v)$

A-B ON THE EXAMPLE



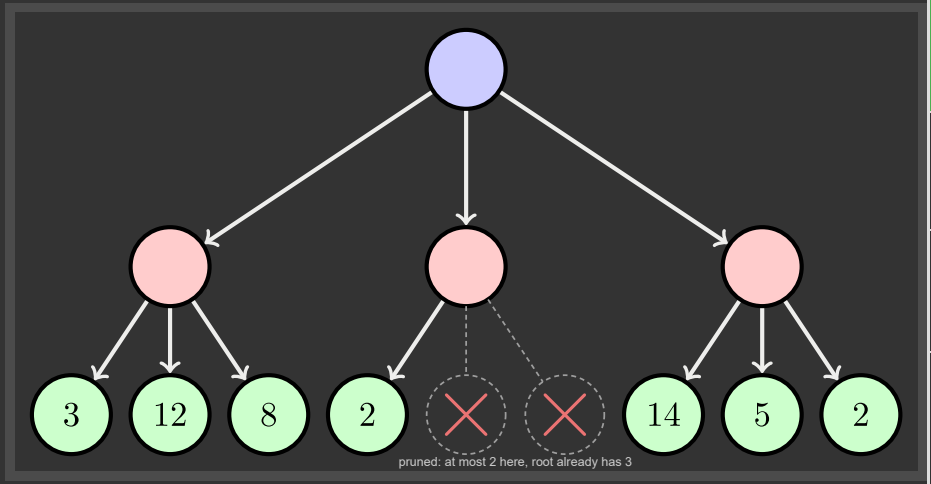
Call	(α, β) received	Children seen	Returns

A-B ON THE EXAMPLE



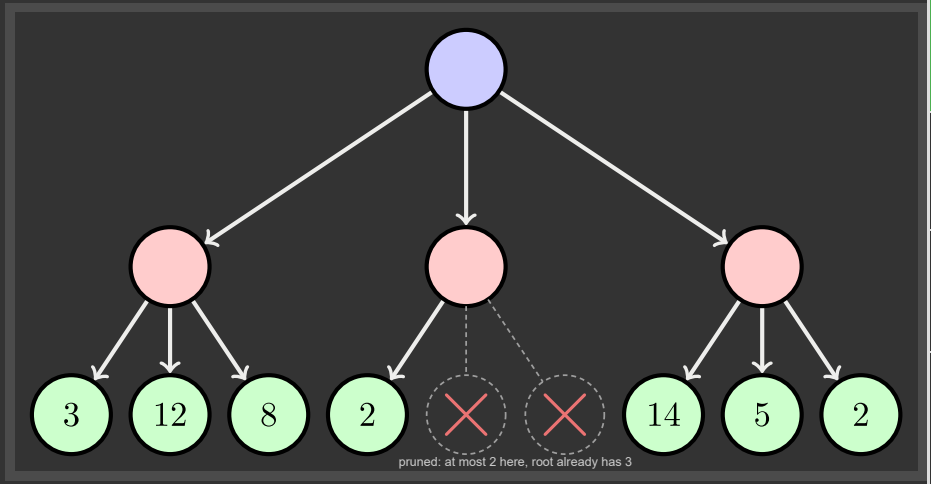
Call	(α, β) received	Children seen	Returns
root, MAX	$(-\infty, +\infty)$		

A-B ON THE EXAMPLE



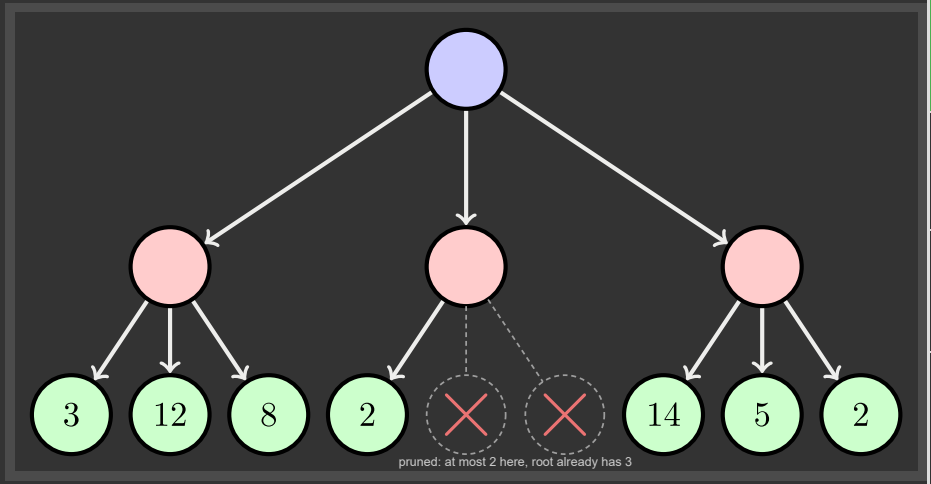
Call	(α, β) **received**	**Children seen**	**Returns**
root, MAX	$(-\infty, +\infty)$		
left MIN	$(-\infty, +\infty)$	3, 12, 8: $v = \mathbf{3}$, never $\leq \alpha$	3; root sets $\alpha = \mathbf{3}$

A-B ON THE EXAMPLE



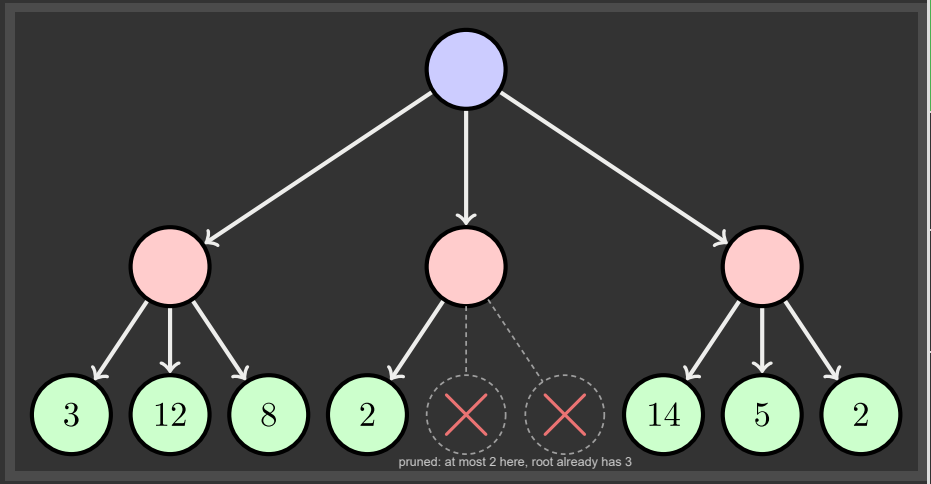
Call	(α, β) **received**	**Children seen**	**Returns**
root, MAX	$(-\infty, +\infty)$		
left MIN	$(-\infty, +\infty)$	3, 12, 8: $v = 3$, never $\leq \alpha$	3; root sets $\alpha = 3$
middle MIN	$(3, +\infty)$	2: $v = 2 \leq \alpha$	2 at once: the other two children are never generated

A-B ON THE EXAMPLE



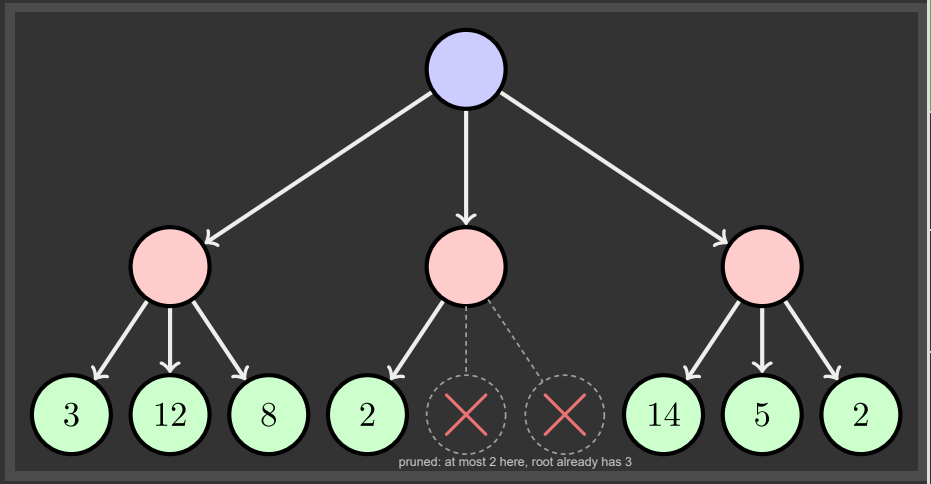
Call	(α, β) received	Children seen	Returns
root, MAX	$(-\infty, +\infty)$		
left MIN	$(-\infty, +\infty)$	3, 12, 8: $v = 3$, never $\leq \alpha$	3; root sets $\alpha = 3$
middle MIN	$(3, +\infty)$	2: $v = 2 \leq \alpha$	2 at once: the other two children are never generated
right MIN	$(3, +\infty)$	14, 5, 2: v stays above 3 until the last child	2; nothing to prune

A-B ON THE EXAMPLE



Call	(α, β) **received**	**Children seen**	**Returns**
root, MAX	$(-\infty, +\infty)$		
left MIN	$(-\infty, +\infty)$	3, 12, 8: $v = 3$, never $\leq \alpha$	3; root sets $\alpha = 3$
middle MIN	$(3, +\infty)$	2: $v = 2 \leq \alpha$	2 at once: the other two children are never generated
right MIN	$(3, +\infty)$	14, 5, 2: v stays above 3 until the last child	2; nothing to prune
root		$\max(3, 2, 2)$	3: move left

A-B ON THE EXAMPLE



Call	(α, β) received	Children seen	Returns
root, MAX	$(-\infty, +\infty)$		
left MIN	$(-\infty, +\infty)$	3, 12, 8: $v = 3$, never $\leq \alpha$	3; root sets $\alpha = 3$
middle MIN	$(3, +\infty)$	2: $v = 2 \leq \alpha$	2 at once: the other two children are never generated
right MIN	$(3, +\infty)$	14, 5, 2: v stays above 3 until the last child	2; nothing to prune
root		$\max(3, 2, 2)$	3: move left

Had the middle leaves come as 6, 4, 2, all three would have been seen: pruning depends on the order.

A-B PRUNING PROPERTIES

A-B PRUNING PROPERTIES

This pruning has no effect on minimax value computed for the root.



A-B PRUNING PROPERTIES

This pruning has no effect on minimax value computed for the root.

Values of intermediate nodes might be wrong

- Important: children of the root may have the wrong value
- So the most naïve version won’ t let you do action selection



A-B PRUNING PROPERTIES

This pruning has no effect on minimax value computed for the root.

Values of intermediate nodes might be wrong

- Important: children of the root may have the wrong value
- So the most naïve version won’t let you do action selection

Good child ordering improves effectiveness of pruning



A-B PRUNING PROPERTIES

This pruning has no effect on minimax value computed for the root.

Values of intermediate nodes might be wrong

- Important: children of the root may have the wrong value
- So the most naïve version won't let you do action selection

Good child ordering improves effectiveness of pruning

With "perfect ordering":

- Time complexity drops to $\mathcal{O}(b^{\frac{m}{2}})$
- Doubles solvable depth
- Full search of, e.g. chess, is still hopeless . . .



A-B PRUNING PROPERTIES

This pruning has no effect on minimax value computed for the root.

Values of intermediate nodes might be wrong

- Important: children of the root may have the wrong value
- So the most naïve version won't let you do action selection

Good child ordering improves effectiveness of pruning

With "perfect ordering":

- Time complexity drops to $\mathcal{O}(b^{\frac{m}{2}})$
- Doubles solvable depth
- Full search of, e.g. chess, is still hopeless . . .

This is a simple example of metareasoning (computing about what to compute)



CONSTRAINED RESOURCES

Problem: In realistic games, cannot search to leaves.



CONSTRAINED RESOURCES

Problem: In realistic games, cannot search to leaves.

Solution: Depth-limited search

- Instead, search only to a limited depth in the tree
- Replace terminal utilities with an evaluation function for non-terminal positions



CONSTRAINED RESOURCES

Problem: In realistic games, cannot search to leaves.

Solution: Depth-limited search

- Instead, search only to a limited depth in the tree
- Replace terminal utilities with an evaluation function for non-terminal positions

Example:

- Suppose we have 100 seconds, can explore 10K nodes / sec
- So can check 1M nodes per move
- α-β reaches about depth 8 – decent chess program



CONSTRAINED RESOURCES

Problem: In realistic games, cannot search to leaves.

Solution: Depth-limited search

- Instead, search only to a limited depth in the tree
- Replace terminal utilities with an evaluation function for non-terminal positions

Example:

- Suppose we have 100 seconds, can explore 10K nodes / sec
- So can check 1M nodes per move
- α-β reaches about depth 8 – decent chess program

Guarantee of optimal play is gone

More depth makes a BIG difference

Use iterative deepening for an anytime algorithm



SUMMARY: ADVERSARIAL SEARCH

	Minimax	α-β pruning	Depth-limited + evaluation

SUMMARY: ADVERSARIAL SEARCH

	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function

SUMMARY: ADVERSARIAL SEARCH

	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function
Optimal against a perfect opponent?	Yes	Yes	No guarantee



SUMMARY: ADVERSARIAL SEARCH

	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function
Optimal against a perfect opponent?	Yes	Yes	No guarantee
Time	$\mathcal{O}(b^m)$	$\mathcal{O}(b^{m/2})$ with perfect move ordering	$\mathcal{O}(b^d)$, choose d to fit the clock

SUMMARY: ADVERSARIAL SEARCH

	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function
Optimal against a perfect opponent?	Yes	Yes	No guarantee
Time	$\mathcal{O}(b^m)$	$\mathcal{O}(b^{m/2})$ with perfect move ordering	$\mathcal{O}(b^d)$, choose d to fit the clock
Space	$\mathcal{O}(bm)$	$\mathcal{O}(bm)$	$\mathcal{O}(bd)$

SUMMARY: ADVERSARIAL SEARCH

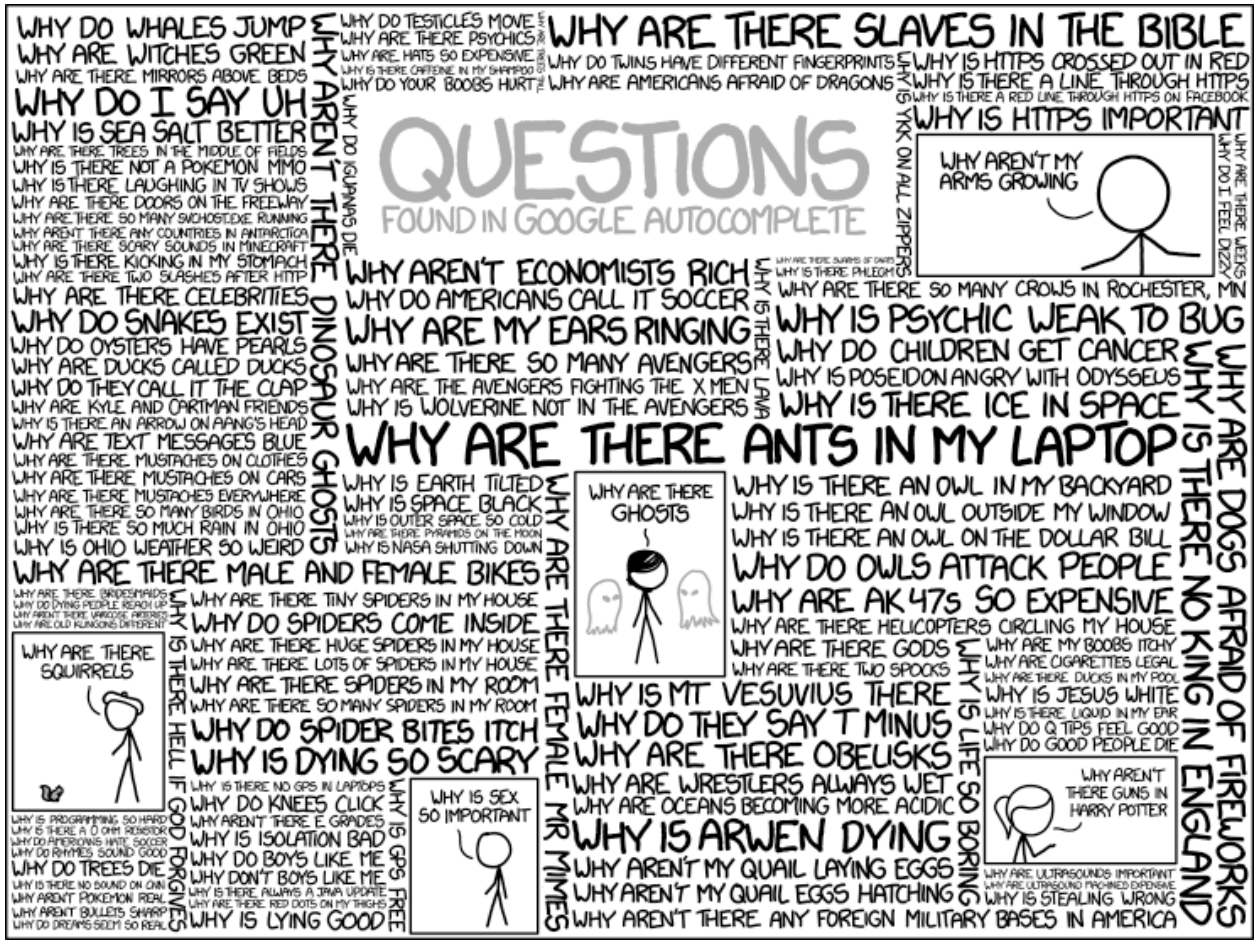
	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function
Optimal against a perfect opponent?	Yes	Yes	No guarantee
Time	$\mathcal{O}(b^m)$	$\mathcal{O}(b^{m/2})$ with perfect move ordering	$\mathcal{O}(b^d)$, choose d to fit the clock
Space	$\mathcal{O}(bm)$	$\mathcal{O}(bm)$	$\mathcal{O}(bd)$
What you give up	Feasible only for tiny games	Nothing; the gain depends on move ordering	Optimality; horizon effects

SUMMARY: ADVERSARIAL SEARCH

	Minimax	**α-β pruning**	**Depth-limited + evaluation**
Answer	Exact minimax value and move	Same value, same move	Approximate; as good as the evaluation function
Optimal against a perfect opponent?	Yes	Yes	No guarantee
Time	$\mathcal{O}(b^m)$	$\mathcal{O}(b^{m/2})$ with perfect move ordering	$\mathcal{O}(b^d)$, choose d to fit the clock
Space	$\mathcal{O}(bm)$	$\mathcal{O}(bm)$	$\mathcal{O}(bd)$
What you give up	Feasible only for tiny games	Nothing; the gain depends on move ordering	Optimality; horizon effects

Chess: $b \approx 35, m \approx 100$. A million nodes per move buys depth 8 with α-β; iterative deepening makes it anytime.

Q & A


QUESTIONS
FOUND IN GOOGLE AUTOCOMPLETE
WHY ARE THERE SLAVES IN THE BIBLE
WHY DO WHALES JUMP
WHY ARE WITCHES GREEN
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SWCHOSTJDXE RUNNING
WHY ARENT THERE ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY ARE THERE MUSTACHES ON CARS
WHY ARE THERE MUSTACHES EVERYWHERE
WHY ARE THERE SO MANY BIRDS IN OHIO
WHY IS THERE SO MUCH RAIN IN OHIO
WHY IS OHIO WEATHER SO WEIRD
WHY ARE THERE MALE AND FEMALE BIKES
WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
WHY AREN'T THERE VARICOSE AFFERETS
WHY ARE OLD KLINGONS DIFFERENT
WHY ARE THERE SQUIRRELS
WHY IS PROGRAMMING SO HARD
WHY IS THERE A 0 OHM RESISTOR
WHY DO AMERICANS HATE SOCCER
WHY DO RHYMES SOUND GOOD
WHY DO TREES DIE
WHY IS THERE NO SOUND ON OVA
WHY AREN'T POKEMON REAL
WHY AREN'T BULLETS SHARP
WHY DO DREAMS SEEM SO REAL
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPIDER BITES ITCH
WHY IS DYING SO SCARY
WHY IS THERE NO GPS IN LAPTOPS
WHY DO KNEES CLICK
WHY AREN'T THERE E GRADES
WHY IS ISOLATION BAD
WHY DO BOYS LIKE ME
WHY DON'T BOYS LIKE ME
WHY IS THERE ALWAYS A JAVA UPDATE
WHY ARE THERE RED DOTS ON MY THIGHS
WHY IS LYING GOOD
WHY DO TESTICLES MOVE
WHY ARE THERE PSYCHICS
WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY DO YOUR BOOBS HURT
WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE AMERICANS AFRAID OF DRAGONS
WHY IS HTTPS CROSSED OUT IN RED
WHY IS THERE A LINE THROUGH HTTPS
WHY IS THERE A RED LINE THROUGH HTTPS ON FACEBOOK
WHY IS HTTPS IMPORTANT
WHY AREN'T MY ARMS GROWING
WHY ARE THERE SWANS AT DAWN
WHY IS THERE PHLEGM
WHY ARE THERE SO MANY CROWS IN ROCHESTER, MN
WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY ARE THERE SO MANY AVENGERS
WHY ARE THE AVENGERS FIGHTING THE X MEN
WHY IS WOLVERINE NOT IN THE AVENGERS
WHY IS PSYCHIC WEAK TO BUG
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE
WHY ARE THERE ANTS IN MY LAPTOP
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY ARE THERE GHOSTS
WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE AK 47s SO EXPENSIVE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOOKS
WHY IS MT VESUVIUS THERE
WHY DO THEY SAY T MINUS
WHY ARE THERE OBELISKS
WHY ARE WRESTLERS ALWAYS WET
WHY ARE OCEANS BECOMING MORE ACIDIC
WHY IS ARWEN DYING
WHY AREN'T MY QUAIL LAYING EGGS
WHY AREN'T MY QUAIL EGGS HATCHING
WHY AREN'T THERE ANY FOREIGN MILITARY BASES IN AMERICA
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO GOOD PEOPLE DIE
WHY AREN'T THERE GUNS IN HARRY POTTER
WHY ARE ULTRASOUNDS IMPORTANT
WHY ARE ULTRASOUND MACHINES EXPENSIVE
WHY IS STEALING WRONG
WHY IS SEX SO IMPORTANT
WHY ARE THERE FEMALE MR MIMES
WHY IS THERE HELL IF GOD FORGIVES
WHY IS GPS FREE
WHY IS LIFE SO BORING
WHY ARE THERE DINOSAUR GHOSTS
WHY AREN'T THERE
WHY DO IGUANAS DIE
WHY IS YKK ON ALL ZIPPERS
WHY ARE DOGS AFRAID OF FIREWORKS
WHY IS THERE NO KING IN ENGLAND
WHY ARE THERE WEEKS
WHY DO I FEEL DIZZY

XKCD